Communication Protocols Manual

for Ezairo® 7100

M-20790-008
May 2016

© SCILLC, 2016
Previous Edition © 2014
"All Rights Reserved"

ON Semiconductor®

Communication Protocols Manual for Ezairo 7100

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction . . . 6
   1.1 Purpose . . . 6
   1.2 Intended Audience . . . 6
   1.3 Conventions . . . 6
   1.4 Manual Organization . . . 6
   1.5 Further Reading . . . 6
2. Algorithm Protection . . . 8
   2.1 Overview . . . 8
   2.2 Restricted and Unrestricted Access Modes . . . 8
      2.2.1 Direct Configuration . . . 8
      2.2.2 JumpROM Configuration . . . 9
   2.3 Protected Resources . . . 9
      2.3.1 Protected Resources in Non-Volatile Storage . . . 9
      2.3.2 Protected Resources in Memory . . . 9
   2.4 File System Security Support . . . 9
3. JumpROM Function Set . . . 11
   3.1 Background . . . 11
   3.2 JumpROM NOP . . . 11
   3.3 JumpROM GetVersion . . . 11
   3.4 JumpROM Reset . . . 11
   3.5 JumpROM Unlock . . . 12
   3.6 JumpROM Wipe . . . 12
   3.7 JumpROM GetError . . . 12
   3.8 JumpROM BootApp . . . 13
   3.9 JumpROM Lock . . . 13
4. Debug Port Protocols . . . 14
   4.1 Introduction . . . 14
   4.2 Basic Protocol Information . . . 14
      4.2.1 CFX Debug Port Addressing . . . 14
      4.2.2 ARM Cortex-M3 Processor Debug Port Addressing . . . 15
      4.2.3 Acknowledgements . . . 15
      4.2.4 Command Format . . . 15
      4.2.5 Response Format . . . 16
      4.2.6 CFX Modes . . . 17
      4.2.7 ARM Cortex-M3 Processor Modes . . . 18
      4.2.8 Debug Ports Overview . . . 18
   4.3 CFX Debug Port Protocol Commands and Responses . . . 19
      4.3.1 Initialize Transmit State . . . 19
      4.3.2 CFX Debug Port State and Information Commands and Responses . . . 19
         4.3.2.1 NOP Command . . . 19
         4.3.2.2 Read Status Command . . . 20
         4.3.2.3 Status Transmit State . . . 20
         4.3.2.4 Write CFX Address Command . . . 21

4.3.2.5 Write ARM Cortex-M3 Processor Debug Address Command . . . 21
4.3.2.6 Acquire CFX Address Command . . . 22
4.3.2.7 Reset CFX Debug Port Command . . . 22
4.3.2.8 Reset ARM Cortex-M3 Processor Debug Port Command . . . 22
4.3.2.9 Set Monitor Reset Bit Command . . . 22
4.3.2.10 Read System ID Command . . . 23
4.3.2.11 CFX SysID Transmit State . . . 23
4.3.2.12 Read and Reset CRC Command . . . 23
4.3.2.13 CRC Transmit State . . . 24
4.3.2.14 Reset and Start Cycle Counter Command . . . 24
4.3.2.15 Get Cycle Count Command . . . 24
4.3.2.16 Cycle Count Transmit State . . . 25
4.3.3 Memory Access Commands and Responses . . . 25
4.3.3.1 Transfer Mode . . . 25
4.3.3.2 Read Memory Command . . . 26
4.3.3.3 Memory Transmit State . . . 26
4.3.3.4 Write Memory Command . . . 27
4.3.4 Register Access Commands and Responses . . . 27
4.3.4.1 Read Normal Registers Command . . . 28
4.3.4.2 Normal Registers Transmit State . . . 29
4.3.4.3 Write Normal Registers Command . . . 29
4.3.4.4 Read Special Registers Command . . . 30
4.3.4.5 Special Registers Transmit State . . . 31
4.3.4.6 Write Special Registers Command . . . 31
4.3.4.7 Read Debug Port Register Command . . . 32
4.3.4.8 Debug Port Register Transmit State . . . 32
4.3.4.9 Write Debug Port Register Command . . . 32
4.3.5 Run Control Commands . . . 32
4.3.5.1 Start CFX Core Command . . . 32
4.3.5.2 Start CFX and ARM Cortex-M3 Cores Command . . . 33
4.3.5.3 Stop CFX Core Command . . . 33
4.3.5.4 Stop CFX and ARM Cortex-M3 Cores Command . . . 33
4.3.5.5 CFX Single-Step Command . . . 33
4.3.5.6 CFX Single-Step With Interrupts Command . . . 33
4.3.5.7 Execute Instruction Command . . . 34
4.3.5.8 JumpROM Command . . . 34
4.3.6 Breakpoint Commands and Responses . . . 34
4.3.6.1 Set Hardware Breakpoints Command . . . 34
4.3.6.2 Write Breakpoint Config Command . . . 35
4.3.6.3 Read Breakpoints Command . . . 36
4.3.6.4 Breakpoints Transmit State . . . 36
4.4 ARM Cortex-M3 Core Debug Port Protocol Commands and Responses . . . 36
4.4.1 Initialize Transmit State . . . 36
4.4.1.1 Quit Debug Command . . . 36
4.4.2 ARM Cortex-M3 Processor Debug Port State and Information Commands and Responses . . . 37

4.4.2.1 Read Status Command . . . 37
4.4.2.2 Status Transmit State . . . 37
4.4.2.3 Enable Debug Command . . . 38
4.4.2.4 Disable Debug Command . . . 38
4.4.2.5 Read System ID Command . . . 38
4.4.2.6 ARM Cortex-M3 Processor SysID Transmit State . . . 39
4.4.2.7 AHB-AP Overrun Clear Command . . . 40
4.4.2.8 Read and Reset CRC Command . . . 41
4.4.2.9 CRC Transmit State . . . 41
4.4.3 Memory Access Commands and Responses . . . 41
4.4.3.1 ARM Cortex-M3 Processor Memory Transfer Mode . . . 41
4.4.3.2 ARM Cortex-M3 Processor Read Memory Command . . . 42
4.4.3.3 ARM Cortex-M3 Processor Memory Transmit State . . . 43
4.4.3.4 ARM Cortex-M3 Processor Write Memory Command . . . 43
4.4.4 Register Access Commands and Responses . . . 44
4.4.4.1 ARM Cortex-M3 Processor Read Core Register Command . . . 44
4.4.4.2 ARM Cortex-M3 Processor Register Transmit State . . . 45
4.4.4.3 ARM Cortex-M3 Processor Write Core Register Command . . . 46
4.4.4.4 Read AHB-AP Register Command . . . 46
4.4.4.5 AHB-AP Register Transmit State . . . 47
4.4.4.6 Write AHB-AP Register Command . . . 47
4.4.5 Run Control Commands . . . 47
4.4.5.1 Start ARM Cortex-M3 Core Command . . . 47
4.4.5.2 Stop ARM Cortex-M3 Core Command . . . 48
4.4.5.3 ARM Cortex-M3 Processor Single-Step Command . . . 48
4.4.5.4 ARM Cortex-M3 Processor Single-Step with Interrupts Command . . . 48
4.4.6 Breakpoint Commands and Responses . . . 48
4.4.6.1 Coordinated Breakpoints Configuration Command . . . 48
A. JumpROM Function Quick Reference . . . 50
B. CFX Debug Port Protocol Quick Reference . . . 51
C. ARM Cortex-M3 Core Debug Port Protocol Quick Reference . . . 56

CHAPTER 1
Introduction

1.1 Purpose

This manual provides information regarding the communication protocols for application developers working with chips based on the CFX Digital Signal Processor (DSP) and the ARM® Cortex-M3 processor. This manual discusses the built-in method of communicating with these chips: the debug port protocol.

This manual further explains the system algorithm protection feature, and how this feature affects the debug port protocol in its efforts to protect potentially sensitive intellectual property (IP) in the form of application code and data.

1.2 Intended Audience

This manual is intended for engineers and anyone else responsible for implementing a communications infrastructure communicating with chips based on the CFX DSP and ARM Cortex-M3 processor. This manual assumes that you know how to use the interfaces, have a general familiarity with communication protocols, and are familiar with the chips. If you are going to use the debug port protocol, it is helpful to have some familiarity with the CFX DSP and ARM Cortex-M3 core instruction sets.

1.3 Conventions

This manual uses the following conventions:

- Any configuration registers, macros and CFX instructions are shown in a `special font`.
- Debug port commands are in **bold** typeface.
- Debug transmit states are in *italic* typeface.
- In general, numbers are presented in decimal notation. In cases where hexadecimal or binary notation is more convenient, these numbers are identified by the prefixes “0x” and “0b” respectively. For example, the decimal number 123456 can also be represented as 0x1E240 or 0b11110001001000000.

1.4 Manual Organization

The *Communication Protocols Manual for Ezairo 7100* contains the following chapters and appendices:

- **Chapter 1: Introduction**, describes the purpose of the manual, the intended audience, conventions used in this manual, outlines how the book is organized, and gives a list of suggested reading for more information.
- **Chapter 2: Algorithm Protection** provides an overview of the algorithm protection features and how the various components of the algorithm protection scheme work together.
- **Chapter 3: JumpROM Function Set** contains a detailed description of the JumpROM functions, including the preconditions for commands and their relation to algorithm protection.
- **Chapter 4: Debug Port Protocols** contains a detailed description of the built-in debug port protocol, including system state information and details on the functionality of each of the commands in the protocol.
- **Appendix A: JumpROM Function Quick Reference** provides a quick reference to the JumpROM functions.
- **Appendix B: CFX Debug Port Protocol Quick Reference** provides a quick reference to the commands available through the debug port protocol.
- **Appendix C: ARM Cortex-M3 Core Debug Port Protocol Quick Reference** provides a quick reference to the commands available through the ARM Cortex-M3 core debug port protocol.

1.5 Further Reading

For more information, refer to the following documents:

- *Getting Started Guide for Ezairo 7100*
- *Hardware Reference Manual for Ezairo 7100*
- *Firmware Reference Manual for Ezairo 7100*
- *Evaluation and Development Board Manual for Ezairo 7100*
- *CFX DSP Architecture Manual*
- *The I2C-Bus Specification*, Philips Semiconductor, document order number 9398 393 40011
- Yiu, Joseph *The Definitive Guide to the ARM Cortex-M3* (Newnes, December 2009) or a similar book describing the ARM Cortex-M3 processor architecture
- *Cortex-M3 Technical Reference Manual*, available from http://infocenter.arm.com/help/index.jsp

CHAPTER 2
Algorithm Protection

2.1 Overview

The algorithm protection scheme provides a mechanism for protecting intellectual property in the form of application code and data stored in memory and non-volatile memory resources.

Upon startup or following a reset, the algorithm protection scheme is automatically set to protect system resources, restricting all memory access through the debug port. Additionally, through support for the use of file system encryption in the non-volatile storage area, the application code and data that is stored in the non-volatile storage area can be protected even when the device package does not physically prevent direct access to the non-volatile storage medium.

2.2 Restricted and Unrestricted Access Modes

The chips operate in one of two different access modes: restricted access mode and unrestricted access mode.

- When operating in restricted access mode, external access to application code or data executing on the device is not available through the debug port interface as certain debug port commands are disabled. A summary of debug port commands and restrictions based on access mode can be found in Appendix B, “CFX Debug Port Protocol Quick Reference” on page 51.
- When operating in unrestricted access mode, external access to application code or data executing on the device is available as all debug port interface commands are available.

To query the current debug port status, check the security mode bit from the debug port status word. For details on querying and interpreting the debug port status, see Section 4.3.2, “CFX Debug Port State and Information Commands and Responses” on page 19.

To change the access mode, use one of the following methods:

- The application running on the CFX DSP core can configure the mode directly, as described in Section 2.2.1, “Direct Configuration”.
- Through the debug port JumpROM commands, the debug host can configure the mode externally, as described in Section 2.2.2, “JumpROM Configuration”.
- If an application boots the device through the debug port instead of a non-volatile storage device attached to the SPI interface, the boot ROM can set the access mode to unrestricted as follows:
  - Specify the boot address as 0x0000 or 0xFFFF or provide an invalid boot signature (data at 0x00000000 of the EEPROM set to something other than 0x5A).
  - Set the key area to all zeros or all ones.

NOTE: If no EEPROM is attached to the device, you can pull the `DIO2` (which the Program ROM configures as SPI_SERI) pin to ground or up to the same level as `VDDO1` to ensure that all zeros or all ones are read from the device for both the boot address and key area.

IMPORTANT: If the application running on the CFX DSP core implements a data transfer mechanism using other external interfaces, such as the general purpose I^2C interface, the application is responsible for limiting access to application code or data to protect intellectual property.

2.2.1 Direct Configuration

The access mode can be configured using the `DEBUG_CFG_ACCESS_MODE` bit in the `D_DEBUG_CFG` register. The `Sys_DebugPort_Set_AccessMode` macro provides a standard method of updating the current access mode.

The contents of memory or non-volatile storage do not need to be cleared when directly switching between access modes.

2.2.2 JumpROM Configuration

The system access mode can be reconfigured by an external device through debug port commands using the JumpROM command set.

To select restricted access mode, execute a JumpROM lock function. No restrictions are placed on the contents of memory or non-volatile storage when executing this function. For more details on the JumpROM lock function, see Section 3.9, “JumpROM Lock” on page 13.

To select unrestricted access mode, use JumpROM functions to clear the protected resources in non-volatile storage and memory, and update the access mode:

1. Run the JumpROM wipe function to clear the protected resources in non-volatile storage. Details of protected resources in non-volatile storage are provided in Section 2.3.1, “Protected Resources in Non-Volatile Storage”. For more details on the JumpROM wipe function, see Section 3.6, “JumpROM Wipe” on page 12.
2. Run the JumpROM unlock function to clear the protected resources in memory and select unrestricted access mode. Details of protected resources in memory are provided in Section 2.3.2, “Protected Resources in Memory”. For more details on the JumpROM unlock function, see Section 3.5, “JumpROM Unlock” on page 12.

NOTE: You can use the JumpROM unlock function if non-volatile storage is not attached to the device, or if the non-volatile storage has already been cleared. However, the JumpROM wipe function does not take much time in either of these cases, so we recommend that you run this function in all cases.

2.3 Protected Resources

2.3.1 Protected Resources in Non-Volatile Storage

The bootloader uses AES keys stored in the key area of the non-volatile (EEPROM) file system to decrypt any encrypted segments of the file system. This key area is protected by algorithm protection and must be cleared to 0xFFFF before the debug port can be externally unlocked. No other areas of the file system are protected.

For more information on the key area and encryption support, see the *Firmware Reference Manual for Ezairo 7100* and Section 2.4, “File System Security Support” on page 9.

2.3.2 Protected Resources in Memory

You can protect data and code from external access to pre-defined sections of memory. When unlocking the debug port externally using the JumpROM functions, use the unlock command to clear these sections of memory before switching to unrestricted access mode. A list of protected memory resources is provided in the *Hardware Reference Manual for Ezairo 7100* for your product.

2.4 File System Security Support

You can use encrypted data in the non-volatile memory to provide protection to software intellectual property for copyright and related purposes.

File system security is provided through the support of AES encrypted data, and as a result, a number of places in the firmware and program ROM support file system encryption. The components of this support are as follows:

- AES decryption implementation
- EEPROM file system
- Bootloader
- Non-volatile image creation tools

Documentation of some of these components has been provided in the appropriate sections of other manuals. Relevant points include:

- The Access Mode Security Blocks section in the CFX Peripherals chapter of the *Hardware Reference Manual for Ezairo 7100* outlines how security is provided, and presents an overview of the hardware components of this secure file system implementation.
- Chapter 4, “Debug Port Protocols” on page 14 of this manual provides a complete description of the restrictions placed on access through the debug port.
- The Program ROM function tables section provided in the *Firmware Reference Manual for Ezairo 7100* provides links to the ROM-based implementations of the AES decryption functions.
- Program ROM support for AES and the decryption of encrypted file system components is documented in the file system decryption support section of the *Firmware Reference Manual for Ezairo 7100*.
- The EEPROM file system security section and application encrypted sections section from the *Firmware Reference Manual for Ezairo 7100* describe the EEPROM file system components.
- The Bootloader operation section of the *Firmware Reference Manual for Ezairo 7100* describes the support in the bootloader for decryption (and the boot errors table lists the `BOOT_ERR_DECRYPT` error code that the bootloader lists in case of a decryption error).
- The Non-Volatile Memory Manager (NVMM) tool section in the *CFX Toolchain Reference* describes the support for creating encrypted (or partially encrypted) non-volatile images.

CHAPTER 3
JumpROM Function Set

3.1 BACKGROUND

The Program ROM contains functions that are executed through the debug port's JumpROM command. These functions provide access to information such as the Program ROM's version and error codes. Security functions for locking the device or wiping and unlocking the device are available. Functions are also provided to reset the system and to boot a selected application from an EEPROM attached to the device.

JumpROM functions all provide the same calling interface. This includes:

- A 16-bit argument passed as part of the JumpROM debug port command
- An optional 24-bit output stored to the debug port data register (P:)
- A software breakpoint that is executed when any JumpROM function completes

IMPORTANT: JumpROM functions do not maintain the current system state. As a result, when a JumpROM function finishes (with the exception of the JumpROM reset function), control passes to a wait loop in the JumpROM handler.

Details on the debug port protocol command set are provided in Section 4.3.5.8, "JumpROM Command" on page 34.

The following sections provide details on each of the functions in the JumpROM function set. A reference table listing the JumpROM commands is available in Appendix A, "JumpROM Function Quick Reference" on page 50.

3.2 JUMPROM NOP

Function Index	Input Description	Output Description
0x00	Not used	None

The JumpROM NOP function does not perform any function. It can be used to establish a connection to the external system.

3.3 JUMPROM GETVERSION

Function Index	Input Description	Output Description
0x01	Not used	Program ROM version number

The JumpROM GetVersion function can be used to check the Program ROM version. The version is returned in the debug register.

3.4 JUMPROM RESET

Function Index	Input Description	Output Description
0x02	Not used	None

The JumpROM Reset function resets the system. The reset is implemented through the watchdog timer. After the reset, the device starts its standard boot sequence and attempts to load the default application.

3.5 JUMPROM UNLOCK

Function Index	Input Description	Output Description
0x03	Not used	None

The JumpROM Unlock function attempts to unlock the debug port. The function first verifies that the protected resources in non-volatile storage are cleared, or that no EEPROM is attached. If the verification is successful, all protected resources in memory are cleared and the debug port is unlocked. For more information on file system security, see the *Firmware Reference Manual for Ezairo 7100*.

3.6 JUMPROM WIPE

Function Index	Input Description	Output Description
0x04	AT_WREN opcode	Timeout status bit for EEPROM writes at bit position SR_X_ZERO (0 - No timeout, 1 - Timeout)

The JumpROM Wipe function attempts to clear the protected resources in non-volatile storage. The function first verifies that an EEPROM is attached. If an EEPROM is detected, it clears the key area, then automatically calls the JumpROM Unlock function. If an EEPROM is not detected, it clears the system memory and unlocks the system. For more information on file system security and algorithm protection schemes, see Chapter 2, “Algorithm Protection” on page 8 and the file system security section of the *Firmware Reference Manual for Ezairo 7100*.

IMPORTANT: For 16-bit addressable EEPROMs, if the first byte of the EEPROM is 0x00 or 0xFF, the JumpROM Wipe function temporarily overwrites this first byte. If a reset occurs before this value is restored, the previous value at this address is lost.

For 16-bit addressable EEPROMs that contain a valid SDA, and for 24-bit addressable EEPROMs, the JumpROM Wipe function does not require any temporary data modification.

3.7 JUMPROM GETERROR

Function Index	Input Description	Output Description
0x06	Not used	System error code

The JumpROM GetError function is used to retrieve the latest system error code. The error code that is currently stored in the `SYSVAR_SYS_ERRNO` global system variable is returned in the debug register. The error codes are generally set during boot time. If the boot fails, more information about the failure can be learned if the error code is known. After a failed boot, the system waits several seconds before rebooting. During that time the error code can be retrieved.

NOTE: As with all JumpROM functions, executing this JumpROM function stops the system from rebooting because control is passed to the JumpROM handler.

See the *Firmware Reference Manual for Ezairo 7100* for further information on Bootloader and BootROM error codes.

3.8 JUMPROM BOOTAPP

Function Index	Input Description	Output Description
0x07	Application to bootload	None

The JumpROM BootApp function is used to force the system to boot the specified application from EEPROM. The application is specified using its integer application number. The current running application (if any) is terminated. The boot sequence is executed, but instead of loading the default application, the parameter to JumpROM BootApp is used to specify which application to load.

If the parameter specifies an invalid application number, the boot will fail. The appropriate error code is generated and the system reboots after a delay. The subsequent boot will use the default application number from the file system.

For more information on application selection, see the *Firmware Reference Manual for Ezairo 7100*.

IMPORTANT: The JumpROM BootApp function cannot be used when using an encrypted Bootloader (see the file system security section of the *Firmware Reference Manual for Ezairo 7100*).

3.9 JUMPROM LOCK

Function Index	Input Description	Output Description
0x09	Not used	None

The JumpROM Lock function is used to unconditionally lock the debug port.

CHAPTER 4
Debug Port Protocols

4.1 INTRODUCTION

The debug port interfaces on the device allows a PC or other system supporting the two-wire Inter-IC (I^2C) protocol to control the CFX DSP and ARM Cortex-M3 core externally. Information on the interfaces and hardware interface components is available in the *Hardware Reference Manual for Ezairo 7100*.

The debug port interfaces implement a protocol layer, hereafter referred to as a *debug port protocol*, that provides the following functionality:

- Direct access to all CFX DSP core registers
- Direct access to all CFX DSP memory and memory-mapped control registers
- Direct access to all ARM Cortex-M3 core and AHB registers
- Direct access to all ARM Cortex-M3 core memory and memory-mapped control registers
- Run control; software, hardware, and external breakpoints; and single stepping capability
- JumpROM function set support

The CFX debug port provides a user with only a peripheral view (through the contents of shared and microcode memory) of the HEAR processing of microcode configurations and phase processing on the FENG. The CFX debug port provides a similar peripheral view of the processing of code executing on the ARM Cortex-M3 processor; however, the ARM Cortex-M3 processor debug port provides more complete access to this core.

In this chapter, the PC or system communicating with the debug port is referred to as the *debug host*. Software running on the CFX DSP is referred to as the *DSP application*. Software running on the ARM Cortex-M3 processor is referred to as the *microcontroller application*.

4.2 BASIC PROTOCOL INFORMATION

The debug port protocols use an I^2C bus interface with an overlaid protocol command set to provide debug access to the CFX DSP core and ARM Cortex-M3 core. The debug port acts as an I^2C slave device. The debug host must act as an I^2C master device.

Handling of debug port commands start with an I^2C start condition and the command is only processed when an I^2C stop condition is encountered. If another I^2C start condition occurs before the receipt of an I^2C stop condition (i.e., an I^2C restart occurs), the debug port ignores all command bytes received prior to the restart.

See *The I^2C-Bus Specification* from Philips Semiconductors for more information.

4.2.1 CFX Debug Port Addressing

The CFX debug port responds to two I^2C addresses:

Internally-programmable
: The DSP application can program the address internally through a register (`D_I2C_CFX_DEBUG_ADDR1`).

Externally-programmable

You can program the address externally using the **Write Address** or **Acquire Address** commands. The DSP application can also program this address internally through a register (`D_I2C_CFX_DEBUG_ADDR2`).

Both of these addresses default to 0x60 on reset, and you can assign them any value between 0x00 to 0x7F.

For more information about I^2C addressing, including typical pitfalls and addressing of the general-purpose I^2C interface, see the *Hardware Reference Manual for Ezairo 7100*.

4.2.2 ARM Cortex-M3 Processor Debug Port Addressing

The ARM Cortex-M3 processor has a debug port that responds to one I^2C address. You can program this address externally using the **Write Cortex-M3 Address** command. The DSP application can also program this address internally through a register (`D_I2C_CM3_DEBUG_ADDR`).

4.2.3 Acknowledgements

The debug port protocol makes use of the I^2C ACK and NAK mechanism for data acknowledgement with the master. After the master has sent the I^2C Address and Read/Write bit, the debug port responds with an ACK by holding the data line low. In a Write transaction, the next byte that the master sends is the command byte. The response of an ACK or a NAK after the command byte depends on the byte received, the system state, and the debug port state.

If one or more of the following conditions is true, the debug port issues a NAK after the command byte:

- The debug port transmit state is *Initialize* and the debug host starts a read request. In this case, the debug port responds with a NAK immediately after the I^2C Address and Read/Write bit.
- The debug port transmit state is *Initialize* and the debug host starts a write request where the command byte is not **Get Status Response**.
- The debug port receives an unknown command.
- The command is not valid in the current security mode (i.e., a command that is not available in restricted mode is attempted while access is restricted).
- The debug port is currently busy (depends on the command).
- The command is not valid in the current CFX DSP core or ARM Cortex-M3 processor run mode (e.g., attempting to read or write registers while the core is running).

After the debug port sends a NAK to the master, it ignores any further data from the master until a new I^2C Start Condition is received.

4.2.4 Command Format

The debug host sends debug port protocol commands to the debug port in I^2C write transfers. The debug port verifies proper transaction framing and format of the command. It also verifies that it is in the proper security and run mode to execute the command. The debug port then deals with executing the command by setting the appropriate states, data registers and control signals.

The general format of a debug port protocol command is shown below. Normal text indicates data sent from the debug host. Bold text indicates responses from the debug port.

`[S] [ADDR][W]` **`[A]`** `[CMDBYTE]` **`[A]`** `[BYTE0]` **`[A]`** `[BYTE1]` **`[A]`** `... [BYTEn]` **`[A]`** `[P]`

Where:

- `[S]` — I²C Start Condition
- `[ADDR]` — 7-bit debug port I²C Address
- `[W]` — Read/Write bit: 0 (Write) for commands
- `[CMDBYTE]` — Debug port protocol command byte
- **`[A]`** — Acknowledgement: ACK or NAK from debug port
- `[BYTEx]` — Byte x of the command sequence
- `[P]` — I²C Stop Condition

After some commands the debug port expects a fixed number of additional bytes. If the master sends more or fewer bytes than are expected before sending the Stop Condition, the debug port ignores the command.

After other commands (including the CFX debug protocol's **Write Memory**, **Write Normal Registers**, **Write Special Registers**, and **Set Hardware Breakpoints** and the ARM Cortex-M3 processor debug protocol's **Write Memory** and **Write Core Registers**), the debug port expects multiple blocks of data, such as memory word values or register values. For these commands, each complete block of data sent by the master is processed once it has been received. If the master sends a partial block of data before the Stop Condition, the debug port ignores the last block of data.

In the descriptions of commands in the following sections, the I²C framing, addressing and acknowledgement data is omitted; the descriptions only show the command and data bytes.

4.2.5 Response Format

The debug port sends a debug port response after the debug host starts an I²C read transfer. The general format of a response is shown below. Normal text indicates data sent from the debug host. Bold text indicates responses from the debug port.

`[S] [ADDR][R]` **`[A] [BYTE0]`** `[A]` **`[BYTE1]`**`[A] ...` **`[BYTEn]`**`[A] [P]`

Where:

- `[S]` — I²C Start Condition
- `[ADDR]` — 7-bit debug port I²C Address
- `[R]` — Read/Write bit: 1 (Read) for responses
- **`[A]`** — Acknowledgement: ACK or NAK from debug port
- **`[BYTEx]`** — Response byte X of the response sequence
- `[A]` — Acknowledgement: ACK or NAK from debug host
- `[P]` — I²C Stop Condition

The contents of the response depend on the transmit state of the debug port, which is determined by the most recent command sent to the debug port.

In the descriptions of responses in the following sections, the I²C framing, addressing and acknowledgement data is omitted; the descriptions only show the data bytes.

4.2.6 CFX Modes

The CFX debug port protocol supports a number of independent modes that affect the execution of instructions on the CFX DSP, the contents of the debug port status response, and the response to debug port commands. These are summarized in Table 1.

Table 1. CFX Debug Port Modes

Mode	State	Description
CFX Run Mode	Running (default)	The CFX DSP is freely executing instructions.
	Stopped	The CFX DSP is not executing instructions because the core has been stopped (e.g., through an explicit stop, software breakpoint, hardware breakpoint, or external breakpoint). You can execute instructions by issuing single-step commands or by putting the debug port into Running mode.
Monitor Reset Mode	Disabled (default)	The Monitor Reset Bit (bit 12) of the debug port status response is 0.
	Enabled	The Monitor Reset BIt (bit 12) of the debug port status response is 1. Send the **Set Monitor Reset Bit** command to enable this mode. This mode is only disabled when the system is reset. You can use this mode for connection management.
Restricted Mode	Unrestricted Mode	The debug port executes all commands normally. The `ACCESS_RESTRICTED` bit is cleared in the `D_DEBUG_CFG` register.
	Restricted Mode (default)	The debug port NAKs some debug port commands, preventing unauthorized access to application code and data. The `ACCESS_RESTRICTED` bit is set in the `D_DEBUG_CFG` register.
Software Breakpoint Mode	Disabled (default)	The CFX DSP treats the `BREAKPOINT` instruction as a `NOP` instruction.
	Enabled	When the CFX DSP executes a `BREAKPOINT` instruction, the DSP is stopped and the status response indicates that a software breakpoint was hit. See Section 4.3.6, “Breakpoint Commands and Responses” on page 34 and the *CFX DSP Architecture Manual* for more information on software breakpoints.
Hardware Breakpoint N Mode (for N=0…7)	Disabled (default)	The CFX DSP does not stop at hardware breakpoint N.
	Enabled	After the CFX DSP executes the instruction at hardware breakpoint N, the DSP is stopped and the status response indicates that a hardware breakpoint was hit. The debug port supports eight hardware breakpoints that can be independently enabled and disabled. See Section 4.3.6, “Breakpoint Commands and Responses” on page 34 for more information on hardware breakpoints.
External Breakpoint Mode	Disabled (default)	The CFX DSP does not stop for external breakpoints.
	Enabled	When the external breakpoint condition is true, the CFX DSP is stopped and the status response indicates that an external breakpoint was hit. See Section 4.3.6, “Breakpoint Commands and Responses” on page 34 and the *Hardware Reference Manual for Ezairo 7100* for more information on external breakpoints.
Coordinated Breakpoint Mode	Disabled (default)	The CFX DSP does not stop for when a breakpoint is encountered on the ARM Cortex-M3 processor. The ARM Cortex-M3 processor does not stop when a breakpoint is encountered on the CFX DSP.
	Enabled	When an ARM Cortex-M3 processor breakpoint is encountered, the CFX DSP is stopped and the status response indicates that an external breakpoint was hit. When a CFX breakpoint is encountered, the ARM Cortex-M3 processor is similarly stopped and a coordinated breakpoint is reported. See Section 4.3.6, “Breakpoint Commands and Responses” on page 34 and the *Hardware Reference Manual for Ezairo 7100* for more information on external breakpoints.

4.2.7 ARM Cortex-M3 Processor Modes

The ARM Cortex-M3 core debug port protocol supports a number of independent modes that affect the execution of instructions on the CFX DSP, the contents of the debug port status response, and the response to debug port commands. These are summarized in Table 2.

Table 2. ARM Cortex-M3 Processor Debug Port Modes

Mode	State	Description
ARM Cortex-M3 Processor Debug Mode	Disabled (default)	The debug support components for the ARM Cortex-M3 processor are disabled.
	Enabled	The debug support components for the ARM Cortex-M3 processor are enabled.
ARM Cortex-M3 Processor Run Mode	Running (default)	The ARM Cortex-M3 processor is freely executing instructions.
	Stopped	The ARM Cortex-M3 processor is not executing instructions because the core has been stopped (e.g., through an explicit stop, software breakpoint, hardware breakpoint, or external breakpoint). You can execute instructions by issuing single-step commands or by putting the debug port into Running mode.
Restricted Mode	Unrestricted Mode	The debug port executes all commands normally. The `ACCESS_RESTRICTED` bit is cleared in the CFX DSP's `D_DEBUG_CFG` register.
	Restricted Mode (default)	The debug port issues NAKs for some debug port commands, preventing unauthorized access to application code and data. The `ACCESS_RESTRICTED` bit is set in the CFX DSP's `D_DEBUG_CFG` register.
Coordinated Breakpoint Mode	Disabled (default)	The CFX DSP does not stop when a breakpoint is encountered on the ARM Cortex-M3 core. The ARM Cortex-M3 core does not stop when a breakpoint is encountered on the CFX DSP.
	Enabled	When an ARM Cortex-M3 processor breakpoint is encountered, the CFX DSP is stopped and the status response indicates that an external breakpoint was hit. When a CFX breakpoint is encountered, the ARM Cortex-M3 core is similarly stopped and a coordinated breakpoint is reported. See Section 4.3.6, "Breakpoint Commands and Responses" on page 34 and the *Hardware Reference Manual for Ezairo 7100* for more information about external breakpoints.

4.2.8 Debug Ports Overview

The CFX and ARM Cortex-M3 core debug port protocols consist of commands that are sent by the debug host to the debug port in I^2C Write transfers. The debug port decodes and executes these commands.

The description of each command lists the next transmit state. For commands that transfer data from the debug port to the debug host (see Table 3), the debug host must perform two I^2C transfers:

1. An I^2C write transfer containing the command and associated data; when the debug port executes the command, it changes its transmit state to the appropriate state.
2. An I^2C read transfer to transfer the data from the debug port to the debug host.

IMPORTANT: The CFX and ARM Cortex-M3 processor debug ports always return to their *Status* transmit state after the next I^2C Read transfer.

Table 3. Commands that Transfer Data from Debug Port to Host

Command	CFX Debug Port	ARM Cortex-M3 Processor Debug Port
Read System ID	X	X
Read Memory	X	X
Read Normal Registers	X	X
Read Special Registers	X	
Read Breakpoints	X	
Read Debug Port Register	X	
Read AHB Registers		X
Read and Reset CRC	X	X

The description of each command indicates the required security mode. Commands that specify “Any” for the security mode can be executed when the debug port is in either restricted or unrestricted mode. Commands that specify “Unrestricted” can only be executed when the debug port is in unrestricted mode. If the debug port receives a command and the debug port is not in the correct security mode, the debug port will not acknowledge the command.

Each description also lists the required run mode. The run mode indicates the required run mode for the core associated with the debug port protocol. To execute some commands, the core must be stopped; other commands can be executed while the core is either running or stopped. The debug port will not acknowledge a command when the run mode is incorrect; see Section 4.2.3, “Acknowledgements” on page 15 for more information.

The CFX debug port protocol commands are summarized in Appendix B, “CFX Debug Port Protocol Quick Reference” on page 51. The ARM Cortex-M3 core debug port protocol commands are summarized in Appendix C, “ARM Cortex-M3 Core Debug Port Protocol Quick Reference” on page 56.

4.3 CFX Debug Port Protocol Commands and Responses

4.3.1 Initialize Transmit State

To guarantee that the debug host can detect when the debug port has reset, the debug port issues NAKs to all read requests until the debug host sends a **Read Status** command to the debug port. The debug port also does not acknowledge the command byte of all write requests other than a **Read Status** command. This is referred to as the *Initialize* transmit state.

The debug port changes to the *Initialize* transmit state when the debug port is reset. The debug port is reset when the system is reset for any reason, or the debug host sends the *Reset Debug Port* command.

4.3.2 CFX Debug Port State and Information Commands and Responses

4.3.2.1 NOP Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x00	`[0x00]`	*Status*	Any	Any

Do nothing.

4.3.2.2 Read Status Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x53 ('S')	`[0x53]`	*Status*	Any	Any

Put the CFX debug port in the *Status* transmit state. For more information, see Section 4.3.2.3, "Status Transmit State".

This must be the first command sent by the debug host after the debug port has reset. For more information, see Section 4.3.1, "Initialize Transmit State".

4.3.2.3 Status Transmit State

Syntax
`[STATUS 15:8] [STATUS 7:0] ... [STATUS 15:8] [STATUS 7:0]`

The CFX debug port status response is the default response to most read requests from the debug host after establishing a connection. For example, sending a **NOP** command followed by a read from the slave returns the debug port status.

The status word consists of two bytes. The status response is repeated indefinitely as long as the master continues to acknowledge the response bytes. This allows the master to poll for a change in the status response using a single I^2C read transfer.

The status response bit fields are listed in Table 4.

Table 4. Debug Port Status Response

Bit	Status Byte Fields	Values[1]
15	High Byte Indicator	1 – Indicates high byte of status response
14	Coordinated Breakpoint Enable Status	0 – Coordinated breakpoints disabled 1 – Coordinated breakpoints enabled
13	I^2C Bus Error Bit[2]	1 – Bus error has occurred 0 – No bus errors detected
12	Monitor Reset Bit	1 – **Set Monitor Reset Bit** command received since last system reset 0 – Default value (cleared by system reset only)
11	Security Mode	0 – Unrestricted mode 1 – Restricted mode
10	External Breakpoint Enable Status	0 – External breakpoints disabled 1 – External breakpoints enabled
9	Hardware Breakpoint Enable Status	0 – All hardware breakpoints disabled 1 – At least one hardware breakpoint enabled
8	Software Breakpoint Enable Status	0 – Software breakpoints disabled 1 – Software breakpoints enabled

Table 4. Debug Port Status Response (Continued)

Bit	Status Byte Fields	Values[1]
7	Low Byte Indicator	0 – Indicates low byte of status response
6	Reserved	0
5	Debug Port Busy	0 – Debug port idle 1 – Debug port busy[3]
4	External Breakpoint Hit Status	0 – External breakpoint not hit 1 – External breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
3	Hardware Breakpoint Hit Status	0 – Hardware breakpoint not hit 1 – Hardware breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
2	Software Breakpoint Hit Status	0 – Software breakpoint not hit 1 – Software breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
1:0	Run state	00 – Running 01 – Stopped 10 – Core halted (SLEEP) 11 - Reserved

1. The bit changes between states when a change in the debug port state is observed unless otherwise specified.

2. The I^2C Bus Error Bit indicates that an I^2C watchdog timeout has occurred. This bit is cleared when the debug port executes the **Reset Debug Port** command, or when the DSP application reads the `D_I2C_ERROR_CNT` register. See the *Hardware Reference Manual for Ezairo 7100* for more information.

3. The debug port is busy if the debug port is blocked from accessing memory due to a CFX and HEAR memory conflict. See the *Hardware Reference Manual for Ezairo 7100* for more information.

4.3.2.4 Write CFX Address Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x41 (‘A’)	`[0x41] [ADDRESS]`	*Status*	Any	Any

Set the externally-programmable CFX debug port I^2C address to the value specified in the argument. Bits 6:0 are used for the address. Bit 7 is ignored since I^2C addresses are only 7 bits.

4.3.2.5 Write ARM Cortex-M3 Processor Debug Address Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x61 (‘a’)	`[0x61] [ADDRESS]`	*Status*	Any	Any

Set the ARM Cortex-M3 processor debug port I^2C address to the value specified in the argument. Bits 6:0 are used for the address. Bit 7 is ignored since I^2C addresses are only 7 bits.

4.3.2.6 Acquire CFX Address Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x04	`[0x04] [MASK]`	*Status*	Any	Any

Set the externally-programmable CFX debug port I^2C address based on the value of the five GPIO inputs and the value of the mask.

A mask is applied when acquiring the address. Bits 4:0 of the programmable address are the bit-wise AND of GPIO[4:0] and MASK[4:0]. Bits 6:5 of the programmable address are copies of MASK[6:5]. Bit 7 of the mask is ignored since I^2C addresses are only 7 bits.

4.3.2.7 Reset CFX Debug Port Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x07	`[0x07]`	*Initialize*	Any	Any

Reset the CFX debug port. Clears the Monitor Reset Bit and breakpoint status bits in the debug port status response, and changes to the *Initialize* transmit state.

4.3.2.8 Reset ARM Cortex-M3 Processor Debug Port Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x51 (‘Q’)	`[0x51]`	*Initialize*	Any	Any

Reset the ARM Cortex-M3 processor debug port. Clears the breakpoint status bits in the debug port status response, and changes to the *Initialize* transmit state.

4.3.2.9 Set Monitor Reset Bit Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x43 (‘C’)	`[0x43]`	*Status*	Any	Any

Set the Monitor Reset Bit in the CFX debug port status response. The Monitor Reset Bit is only cleared by a system reset or the **Reset Debug Port** command.

4.3.2.10 Read System ID Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x49 ('I')	`[0x49]`	*SysID*	Any	Any

Put the CFX debug port in the *SysID* transmit state so that the debug host can read the system identification information. For more information, see Section 4.3.2.11, "CFX SysID Transmit State" on page 23.

4.3.2.11 CFX SysID Transmit State

Syntax
`[DEBUG PROTOCOL MAJOR] [DEBUG PROTOCOL MINOR] [CHIP FAMILY] [CHIP VERSION] [CHIP REVISION MAJOR] [CHIP REVISION MINOR]`

The 6-byte system identification response contains information about the CFX debug port protocol and the chip. Send the **Read System ID** command to set up this response.

This document describes debug port protocol version 2.1 (`[DEBUG PROTOCOL MAJOR]`=1, `[DEBUG_PROTOCOL_MINOR]`=9). If a future change to the debug protocol is backward-compatible with this version then the minor version number is updated. If a future change is not backward-compatible, the major version number is updated and the minor version number reset to 0.

The `[CHIP FAMILY]` is identified in the *Hardware Reference Manual for Ezairo 7100* for your product.

The `[CHIP VERSION]`, `[CHIP REVISION MAJOR]` and `[CHIP REVISION MINOR]` are combined to identify a particular version of a chip. Different chip versions are assigned to identify drastically different and incompatible chips within a chip family. A change to the major chip revision indicates that some functional change has been made to the chip that might affect DSP application code.

A change to the minor chip revision indicates that some internal functional change has been made to the chip that does not affect DSP application code.

Refer to the device identification section of the *Hardware Reference Manual for Ezairo 7100* for more information about major and minor revisions.

4.3.2.12 Read and Reset CRC Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4D ('M')	`[0x4D]`	*CRC*	Any	Any

Put the debug port in the *CRC* transmit state so that the debug host can read the debug port checksum. When the debug port executes this command, it stores the current CRC value to be sent to the debug host and resets the CRC value to 0xFFFF.

Use the **Read and Reset CRC** command to read the current CRC value and reset the CRC to 0xFFFF. The CRC includes all transferred bytes up to and including the **Read and Reset CRC** command. If the debug host reads the CRC using the next I^2C read transfer, the CRC bytes read by the debug host will be included in the next CRC calculation.

The debug port maintains a CRC of all bytes transmitted and received. The CRC is updated automatically after a complete byte has been transferred in either direction. This includes all bytes that are not acknowledged, invalid commands, or commands attempted in the incorrect security or run mode. If a byte is not completely transferred, it is not included in the CRC. The CRC does not include the address byte of I^2C transactions.

The debug port uses CRC-CCITT, which has the parameters described in Table 5.

Table 5. CRC-CCITT Algorithm Parameters

CRC Parameter	Parameter Value
Order	16
Polynomial	$x^{16} + x^{12} + x^5 + 1$
Polynomial (hex)	0x1021
Initial Value (hex)	0xFFFF
Final XOR Value (hex)	0x0000[1]

1. No data manipulation is required (i.e., no data byte reversal, reversal of the final result or other finalization)

For more information, see Section 4.3.2.13, “CRC Transmit State”.

4.3.2.13 CRC Transmit State

Syntax
`[CRC 15:8] [CRC 7:0]`

Transfer the value of the 16-bit debug port CRC when the preceding **Read and Reset CRC** command was executed. For more information, see Section 4.3.2.12, “Read and Reset CRC Command” on page 23.

4.3.2.14 Reset and Start Cycle Counter Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x58 (‘X’)	`[0x58]`	*Status*	Unrestricted	Any

Reset the cycle count to 0 and start the cycle counter. For more information, see Section 4.3.2.16, “Cycle Count Transmit State”.

4.3.2.15 Get Cycle Count Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x56 (‘V’)	`[0x56]`	*Cycle Count*	Unrestricted	Any

Put the debug port in the *Cycle Count* transmit state so that the debug host can read the cycle count. For more information, see Section 4.3.2.16, “Cycle Count Transmit State”.

4.3.2.16 Cycle Count Transmit State

Syntax
`[CYCLE COUNT 15:8] [CYCLE COUNT 7:0]`

The CFX debug port contains a cycle counter that counts CFX DSP core execution cycles while the CFX DSP core is running. When the chip is reset, the cycle count register is reset to 0 and does not count. To reset the cycle count register to 0 and start counting cycles, the debug host must send the **Reset and Start Cycle Counter** command. Use the **Get Cycle Count** command to retrieve the cycle count register’s value. For more information about individual instruction execution cycles, see the *CFX DSP Architecture Manual*.

The cycle count register is 16 bits. When it reaches 0xFFFF, the cycle counter stops counting, and the register remains at 0xFFFF until it is reset.

4.3.3 Memory Access Commands and Responses

For memory transfers, the CFX DSP core must be stopped. The method by which the core was stopped does not matter (e.g., through an explicit stop, software breakpoint, hardware breakpoint or external breakpoint). If the core is not stopped, the commands to read or write memory will not be acknowledged.

4.3.3.1 Transfer Mode

The CFX debug port memory access commands contain a Transfer Mode byte that specifies the memory space, data width, and transfer size for the read or write operation. The Transfer Mode byte is described in Table 6.

Table 6. Transfer Mode

Bit(s)	Description	Values
7:5	Reserved	000
4	Transfer Size	0 - Multiple words 1 - Single word
3:2	Memory Space	00 - Reserved 01 - X memory 10 - Y memory 11 - P memory
1:0	Data Width	00 - 8-bit 01 - 16-bit 10 - 24-bit 11 - 32-bit

Data is transferred with the most significant bytes first; for a 32-bit transfer, the sequence is:

1. Bits 31 to 24
2. Bits 23 to 16
3. Bits 15 to 8
4. Bits 7 to 0

The data width is specified independently from the memory space. If the transfer width is smaller than the actual memory width, the least significant bits are transferred. For example, for a 32-bit memory with an 8-bit transfer mode set, only bits 7 to 0 are transferred. When reading memory, the data is truncated from the actual memory width to the desired transfer width. When writing memory, the data is zero-extended from the transfer width to the actual memory width.

The debug port supports transferring either single words or multiple words. When the debug host is reading multiple words, the debug port queues up the next word to be sent, possibly triggering a side effect. For memory reads that cause side effects, the single word transfer can be used. In this mode, the debug port performs only a single access for the requested word.

When memory is being read in single word mode, subsequent bytes read from the debug port are all zero. Similarly, when memory is being written in single word mode, extra bytes received are ignored.

4.3.3.2 Read Memory Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x52 ('R')	`[0x52] [TRANSFER_MODE] [ADDR 15:8] [ADDR 7:0]`	*Memory*	Unrestricted	Stopped

Reading memory is initiated through the **Read Memory** CFX debug port command.

The `[TRANSFER_MODE]` argument indicates the memory space and data width for the transfer, as described in Section 4.3.3.1, "Transfer Mode" on page 25. The next two bytes specify the high and low bytes of the starting address.

Once the Read Memory command has been sent, the debug port switches to the *Memory* transmit state.

To read the actual memory data, the debug host must start an I^2C read transfer. Depending on the specified data width, the address is automatically incremented after one, two, three, or four bytes have been transferred, and the subsequent word is queued to send.

The debug port performs no special processing on the address. The address automatically wraps around when it reaches the end of the address range (i.e., when the address reaches 0xFFFF, the next word read will be 0x0000).

Below is an example transaction for reading two words from X Memory (0x0010 and 0x0011) using the 24-bit transfer mode. Normal text indicates data sent from the debug host. Bold text indicates responses from the debug port.

```
[S] [ADDR][W] [A] ['R'] [A] [0x6] [A] [0x00] [A] [0x10] [A] [P]
[S] [ADDR][R] [A] [X(0010) 23:16] [A] [X(0010) 15:8] [A] [X(0010) 7:0] [A]
    [X(0011) 23:16] [A] [X(0011) 15:8] [A] [X(0011) 7:0] [A] [P]
```

4.3.3.3 Memory Transmit State

Syntax
`[DATA.0] [DATA.1] ... [DATA.n]`

Transfers data from the CFX debug port to the debug host in response to a **Read Memory** command. For more information, see Section 4.3.3.2, "Read Memory Command".

4.3.3.4 Write Memory Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode

0x57 ('W')	`[0x57] [TRANSFER_MODE]`
	`[ADDR 15:8] [ADDR 7:0]`
	`[DATA.0] [DATA.1] ... [DATA.n]`
		Status	Unrestricted	Stopped

Writing memory is initiated through the Write Memory CFX debug port command.

The `[TRANSFER_MODE]` argument indicates the memory space and data width for the transfer as described in Section 4.3.3.1, "Transfer Mode" on page 25. The next two bytes specify the high and low bytes of the starting address.

The Write Memory transaction occurs in a single I^2C write transfer. After the debug port has acknowledged the memory address, the debug host can begin sending data to be written to memory. Depending on the transfer mode, the debug host sends one, two, three, or four bytes per word. Once the full word has been transmitted, the CFX debug port queues the word for writing to memory and acknowledges the last byte. Hence the debug host can send data to the debug port continuously without the debug port having to stretch the clock. The CFX debug port automatically increments the address by one after each write.

The CFX debug port performs no special processing on the address. The address automatically wraps around when it reaches the end of the address range (i.e., when the address reaches 0xFFFF, the next word read is 0x0000).

Below is an example transaction for writing two words to X Memory (0xABCDEF to 0x0010 and 0x123456 to 0x0011) using the 24-bit transfer mode. Normal text indicates data sent from the debug host. Bold text indicates responses from the debug port.

`[S] [ADDR][W]` **`[A]`** `['W']` **`[A]`** `[0x6]` **`[A]`** `[0x00]` **`[A]`** `[0x10]` **`[A]`** `[0xAB]` **`[A]`** `[0xCD]` **`[A]`** `[0xEF]` **`[A]`** `[0x12]` **`[A]`** `[0x34]` **`[A]`** `[0x56]` **`[A]`** `[P]`

4.3.4 Register Access Commands and Responses

The two categories of CFX DSP core registers are *normal* registers and *special* registers which have core requirements as follows:

- Before you access normal registers, the CFX DSP core must be stopped. The method by which the core is stopped does not matter (e.g., through an explicit stop, software breakpoint, hardware breakpoint, or external breakpoint). If the core is not stopped, the commands to read or write normal registers will not be acknowledged.
- Special registers can be read when the CFX DSP core is running or stopped. The CFX DSP core must be stopped before writing to special registers.

The debug port also contains a special debug port hardware register, `D_DEBUG_DATA`. The DSP application can read or write this register from program memory at P:0xF036. The debug host can always read and write this register whether the CFX DSP core is running or stopped.

4.3.4.1 Read Normal Registers Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x44 ('D')	`[0x44] [REGISTER INDEX]`	*Normal Registers*	Unrestricted	Stopped

Put the CFX debug port in the *Normal Registers* transmit state so that the debug host can read normal registers.

The `[REGISTER INDEX]` argument indicates the starting register index.

The response to this command is a sequence of register values. For each register, the debug port sends 24 bits, most significant byte first:

1. Bits 23 to 16
2. Bits 15 to 8
3. Bits 7 to 0

For registers that are smaller than the transfer width, the upper bits are zero padded. If a register is reserved or ignored, the debug port sends zero.

Once the CFX debug port has sent a register, it increments the register index to transfer the next register. When the register index reaches 255, it wraps back to 0.

The normal register widths and indexes are listed in Table 7.

Table 7. Normal Register Indexes

Index	Register	Width		Index	Register	Width
0	Reserved			32	XCIRC0	15
1	Reserved			33	XCIRC1	15
2	Reserved			34	XCIRC2	15
3	Reserved			35	XCIRC3	15
4	X0	24		36	XCIRC4	15
5	X1	24		37	XCIRC5	15
6	Y0	24		38	XCIRC6	15
7	Y1	24		39	XCIFC7	15
8	XP0	16		40	YCIRC0	15
9	XP1	16		41	YCIRC1	15
10	XP2	16		42	YCIRC2	15
11	XP3	16		43	YCIRC3	15
12	XP4	16		44	Ignored	
13	XP5	16		45	Ignored	
14	XP6	16		46	Ignored	
15	XP7	16		47	SP	16

Table 7. Normal Register Indexes (Continued)

Index	Register	Width		Index	Register	Width
16	XSTEP0	16		48	LR	16
17	XSTEP1	16		49	ILPC	16
18	XSTEP2	16		50	SR	16
19	XSTEP3	16		51	ILSR	16
20	XSTEP4	16		52	A0.L	24
21	XSTEP5	16		53	A1.L	24
22	XSTEP6	16		54	B0.L	24
23	XSTEP7	16		55	B1.L	24
24	YP0	16		56	A0.H	24
25	YP1	16		57	A1.H	24
26	YP2	16		58	B0.H	24
27	YP3	16		59	B1.H	24
28	YSTEP0	16		60	A0.E	8
29	YSTEP1	16		61	A1.E	8
30	YSTEP2	16		62	B0.E	8
31	YSTEP3	16		63	B1.E	8
				64-255	Reserved	

4.3.4.2 Normal Registers Transmit State

Syntax
[REG.0 23:16] [REG.0 15:8] [REG.0 7:0] ... [REG.n 23:16] [REG.n 15:8] [REG.n 7:0]

Transfer the values of normal registers. For more information, see Section 4.3.4.1, “Read Normal Registers Command” on page 28.

4.3.4.3 Write Normal Registers Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x46 (‘F’)	[0x46] [REGISTER INDEX] [REG.0 23:16] [REG.0 15:8] [REG.0 7:0] ... [REG.n 23:16] [REG.n 15:8] [REG.n 7:0]	*Status*	Unrestricted	Stopped

Write values to normal registers.

The [REGISTER INDEX] argument indicates the starting register index.

Subsequent bytes contain a sequence of register values. For each register, the debug host must send 24 bits, most significant byte first:

1. Bits 23 to 16
2. Bits 15 to 8
3. Bits 7 to 0

For registers that are smaller than the transfer width, the upper bits must be zero. If a register is Reserved or Ignored, the debug host must send a value of 0.

Once the CFX debug port has received a register, it increments the register index to transfer the next register. When the register index reaches 255, it wraps back to 0.

The normal register widths and indexes are listed in Table 7 on page 28.

4.3.4.4 Read Special Registers Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x31 ('1')	`[0x31] [REGISTER INDEX]`	*Special Registers*	Unrestricted	Any

Put the CFX debug port in the *Special Registers* transmit state so that the debug host can read special registers.

The `[REGISTER INDEX]` argument indicates the starting register index.

The response to this command is a sequence of register values. For each register, the debug port sends 16 bits, most significant byte first:

1. Bits 15 to 8
2. Bits 7 to 0

For registers that are smaller than the transfer width, the upper bits are zero padded. If a register is reserved or ignored, the debug port sends zero.

Once the debug port has sent a register, it increments the register index to transfer the next register. When the register index reaches 255, it wraps back to 0.

The special register widths and indexes are listed in Table 8.

Table 8. Special Register Indexes

Index	Register	Width		Index	Register	Width
0	LE0	16		8	LC0	12
1	LE1	16		9	LC1	12
2	LE2	16		10	LC2	12
3	LE3	16		11	LC3	12
4	LS0	16		12	LP	3

Table 8. Special Register Indexes (Continued)

Index	Register	Width		Index	Register	Width
5	LS1	16		13	PC (Read-Only)	16
6	LS3	16		14-255	Reserved	
7	LS3	16				

The special registers LEn, LSn, LCn and LP represent the state of the CFX DSP core's hardware loop stack. LP is the loop pointer, and is 0 when no hardware loops are active, or 4, 5, 6 or 7 when 1, 2, 3 or 4 hardware loops are active respectively. LEn, LSn and LCn contain the values of LEA, LSA and LC respectively for each nested hardware loop. See the *CFX DSP Architecture Manual* for more information about hardware loops.

4.3.4.5 Special Registers Transmit State

Syntax
[REG.0 15:8] [REG.0 7:0] ... [REG.n 15:8] [REG.n 7:0]

Transfer the values of special registers. For more information, see Section 4.3.4.4, "Read Special Registers Command".

4.3.4.6 Write Special Registers Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x32 ('2')	[0x32] [REGISTER INDEX] [REG.0 15:8] [REG.0 7:0] ... [REG.n 15:8] [REG.n 7:0]	*Status*	Unrestricted	Stopped

Write values to special registers.

The [REGISTER INDEX] argument indicates the starting register index.

Subsequent bytes contain a sequence of register values. For each register, the debug host must send 16 bits, most significant byte first:

1. Bits 15 to 8
2. Bits 7 to 0

For registers that are smaller than the transfer width, the upper bits must be zero. If a register is Reserved, the debug host must send a value of 0.

Once the CFX debug port has received a register, it increments the register index to transfer the next register. When the register index reaches 255, it wraps back to 0.

The special register widths and indexes are listed in Table 8 on page 30.

You cannot use this command to modify the program counter (PC). Instead, you must single-step the GOTO.0D instruction using the **Execute Instruction** command. See Section 4.3.5.7, “Execute Instruction Command” on page 34 for more information.

4.3.4.7 Read Debug Port Register Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x33 (‘3’)	[0x33]	*Debug Port Register*	Any	Any

Put the debug port in the *Debug Port Register* transmit state so that the debug host can read the debug port register.

The DSP application can read and write the CFX debug port register, D_DEBUG_DATA. The debug host can read this register regardless of run mode or security mode.

4.3.4.8 Debug Port Register Transmit State

Syntax
[DATA 31:24] [DATA 23:16] [DATA 15:8] [DATA 7:0]

Transfer the value of the CFX debug port register. For more information, see Section 4.3.4.7, “Read Debug Port Register Command”.

4.3.4.9 Write Debug Port Register Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x34 (‘4’)	[0x34] [DATA 31:24] [DATA 23:16] [DATA 15:8] [DATA 7:0]	*Status*	Any	Any

Write a value to the CFX debug port register, D_DEBUG_DATA. This register can be read and written by the DSP application, and the debug host can write to this register regardless of run mode or security mode.

4.3.5 Run Control Commands

4.3.5.1 Start CFX Core Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x47 (‘G’)	[0x47]	*Status*	Any	Any

Puts the CFX debug port into Running mode, allowing the CFX DSP core to run freely.

4.3.5.2 Start CFX and ARM Cortex-M3 Cores Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x55 ('U')	`[0x55]`	*Status*	Any	Any

Puts both debug ports into Running mode, allowing the CFX DSP and ARM Cortex-M3 cores to run freely.

4.3.5.3 Stop CFX Core Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x50 ('P')	`[0x50]`	*Status*	Any	Any

Puts the CFX debug port in Stopped mode, stopping the CFX DSP core.

4.3.5.4 Stop CFX and ARM Cortex-M3 Cores Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x59 ('Y')	`[0x59]`	*Status*	Any	Any

Puts both debug ports in Stopped mode, stopping the CFX DSP and ARM Cortex-M3 cores.

4.3.5.5 CFX Single-Step Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x54 ('T')	`[0x54]`	*Status*	Unrestricted	Stopped

Execute an instruction from the DSP application on the CFX DSP core. When the CFX debug port executes this command, the CFX DSP core ignores any pending enabled interrupts.

For instructions with delay slots, this command will execute all instructions in the delay slots.

4.3.5.6 CFX Single-Step With Interrupts Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4E ('N')	`[0x4E]`	*Status*	Unrestricted	Stopped

Execute an instruction from the DSP application on the CFX DSP core. If interrupts are enabled on the CFX core and there is a pending enabled interrupt when the debug port executes this command, the CFX DSP core will execute the interrupt vector operation instead of the instruction at the program counter. After the vector operation, the program counter points to the first instruction of the interrupt handler.

For instructions with delay slots, this command will execute all instructions in the delay slots.

4.3.5.7 Execute Instruction Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4F ('O')	`[0x4F] [INSTR 31:24] [INSTR 23:16] [INSTR 15:8] [INSTR 7:0]`	*Status*	Unrestricted	Stopped

Execute the instruction provided on the CFX DSP core. This command does not modify the program counter (`PC`) unless the instruction modifies `PC`.

4.3.5.8 JumpROM Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4A ('J')	`[0x4A] [CMD 7:0] [ARG 15:8] [ARG 7:0]`	*Status*	Any	Any

This command causes the CFX DSP core to execute the JumpROM function specified. For more information regarding the various JumpROM functions and their functionality, see Chapter 3, "JumpROM Function Set" on page 11.

When the debug port receives this command, it transfers control of the CFX DSP core to the JumpROM function. When the JumpROM function completes, the CFX DSP executes a `BREAKPOINT` instruction within an infinite loop. To execute a JumpROM function, we recommend that the debug host follow these steps:

1. Establish a debug port connection.
2. Stop the CFX DSP core.
3. Send the **Write Breakpoint Config** command to enable software breakpoints.
4. Send the **JumpROM** command to transfer control to the JumpROM function.
5. Poll the status response to detect when the CFX DSP core executes the `BREAKPOINT` instruction.
6. Read the response to the JumpROM command, if any, from the debug port register.

4.3.6 Breakpoint Commands and Responses

4.3.6.1 Set Hardware Breakpoints Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x42 ('B')	`[0x42] [HWBP.0] [ADDR.0 15:8] [ADDR.0 7:0] [HWBP.1] [ADDR.1 15:8] [ADDR.1 7:0] ... [HWBP.n] [ADDR.n 15:8] [ADDR.n 7:0]`	*Status*	Unrestricted	Any

The debug host sends this command to configure hardware breakpoints. Each hardware breakpoint can be individually enabled or disabled. A hardware breakpoint causes the CFX DSP core to stop when it executes the instruction at a specified address.

The debug port supports up to 8 hardware breakpoints. The debug host must send three bytes to configure each hardware breakpoint. The `[HWBP]` byte selects the desired hardware breakpoint and indicates whether it should be enabled or disabled, as described in Table 9. The following two bytes specify the hardware breakpoint address.

Table 9. HWBP.n Fields

Bit(s)	Description	Values
7	Enable/Disable	0 - Disable the hardware breakpoint 1 - Enable the hardware breakpoint
6:3	Reserved	0
2:0	Hardware breakpoint index	0-7

The debug host can configure hardware breakpoints in any order. The debug port configures each hardware breakpoint immediately when it receives the `[HWBP]` byte and two address bytes for that breakpoint.

4.3.6.2 Write Breakpoint Config Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4B ('K')	`[0x4B] [ESWBP] [HWBP]`	*Status*	Any	Any

Send this command to enable and disable software, hardware and external breakpoints. Each type of breakpoint and each individual hardware breakpoint can be enabled or disabled separately. The `[ESWBP]` byte configures software and external breakpoints, as described in Table 10. The `[HWBP]` byte configures hardware breakpoints, as described in Table 11.

Table 10. ESWBP Fields

Bit(s)	Description	Values
7:	Reserved	0
2	Coordinated breakpoints	0 - Disable coordinated breakpoints 1 - Enable coordinated breakpoints
1	External breakpoints	0 - Disable external breakpoints 1 - Enable external breakpoints
0	Software breakpoints	0 - Disable software breakpoints 1 - Enable software breakpoints

Table 11. HWBP Fields

Bit(s)	Description	Values
7	Enable/Disable 7	0 - Disable hardware breakpoint 7 1 - Enable hardware breakpoint 7
...		
0	Enable/Disable 0	0 - Disable hardware breakpoint 0 1 - Enable hardware breakpoint 0

4.3.6.3 Read Breakpoints Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x45 ('E')	`[0x45]`	*Breakpoints*	Unrestricted	Any

Put the debug port in the *Breakpoints* transmit state so that the debug host can read the current breakpoint configuration.

4.3.6.4 Breakpoints Transmit State

Syntax
`[ESWBP] [HWBP] [ADDR.0 15:8] [ADDR.0 7:0] ... [ADDR.7 15:8] [ADDR.7 7:0]`

Transfer the current breakpoint configuration in response to the **Read Breakpoints** command.

The `[ESWBP]` and `[HWBP]` bytes indicate whether software, hardware, external, and coordinated breakpoints are enabled. These bytes are described in Section 4.3.6.2, "Write Breakpoint Config Command" on page 35. The remaining bytes indicate the configured breakpoint addresses for hardware breakpoints 0-7 in ascending order.

4.4 ARM Cortex-M3 Core Debug Port Protocol Commands and Responses

4.4.1 Initialize Transmit State

This is the initial transmit state of the ARM Cortex-M3 processor debug port after power-on reset. The ARM Cortex-M3 processor debug port returns to this transmit state when:

- The ARM Cortex-M3 processor or Ezairo 7100 device is reset due to any system reset (POR, nRESET, etc.)
- The ARM Cortex-M3 processor debug port is reset due to a CFX debug port **Reset ARM Cortex-M3 Processor Debug Port** command, as described in Section 4.3.2.8, "Reset ARM Cortex-M3 Processor Debug Port Command"
- The **Quit Debug** command is received by the debug port, as described in Section 4.4.1.1, "Quit Debug Command"

In Transmit-Initialize state, the ARM Cortex-M3 processor debug port does not respond to a read; a NAK response is sent (after the address and read bit). When the ARM Cortex-M3 processor debug port receives a **Read Status** command (and only that command), the transmit state transitions to Transmit-Status.

4.4.1.1 Quit Debug Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x71 ('q')	`[0x71]`	*Initialize*	Any	Any

Quit the debug port session.

4.4.2 ARM Cortex-M3 Processor Debug Port State and Information Commands and Responses

4.4.2.1 Read Status Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x73 ('s')	`[0x73]`	*Status*	Any	Any

Put the ARM Cortex-M3 processor debug port and debug state in the *Status* transmit state to retrieve status and start a debug port session. For more information, see Section 4.3.2.3, "Status Transmit State".

4.4.2.2 Status Transmit State

Syntax
`[STATUS 15:8] [STATUS 7:0] ... [STATUS 15:8] [STATUS 7:0]`

The ARM Cortex-M3 processor debug port status response is the default response to most read requests from the debug host after establishing a connection, both before and after enabling the debug components.

The status word consists of two bytes. The status response is repeated indefinitely as long as the master continues to acknowledge the response bytes. This allows the master to poll for a change in the status response using a single I^2C read transfer.

The status response bit fields are listed in Table 4.

Table 12. ARM Cortex-M3 Processor Debug Port Status Response

Bit	Status Byte Fields	Values[1]
15	High Byte Indicator	1 – Indicates high byte of status response
14	Coordinated Breakpoint Enable Status	0 – Coordinated breakpoints disabled 1 – Coordinated breakpoints enabled
13:12	Reserved	00
11	Security Mode	0 – Unrestricted mode 1 – Restricted mode
10	ARM Cortex-M3 Processor Lockup Status	0 – The ARM Cortex-M3 processor is not in a lockup state 1 – The ARM Cortex-M3 processor is locked up
9	ARM Cortex-M3 Processor Enable Status	0 – The ARM Cortex-M3 processor is disabled 1 – The ARM Cortex-M3 processor is enabled
8	Reserved	000
7	Low Byte Indicator	0 – Indicates low byte of status response
6	Reserved	0
5	AHB-AP Command Busy	0 – No AHB-AP commands are not active 1 – An AHB-AP command is being executed

Table 12. ARM Cortex-M3 Processor Debug Port Status Response (Continued)

Bit	Status Byte Fields	Values[1]
4	AHB-AP Overrun Status	0 – No AHP-AP command overruns have occurred 1 – An AHP-AP command overrun has occurred
3:2	Reserved	000
1:0	Run state	00 – Running 01 – Stopped 10 – Core halted (sleeping due to WFI, WFE, or Sleep on exit) 11 - Reserved

1. The bit changes between states when a change in the debug port state is observed unless otherwise specified.

4.4.2.3 Enable Debug Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x65 (‘e’)	`[0x65]`	*Status*	Any	Any

Power on the ARM Cortex-M3 processor debug port components. Required for any debug commands except the **Read Status**, **Enable Debug** and **Quit Debug** commands.

4.4.2.4 Disable Debug Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x64 (‘d’)	`[0x64]`	*Status*	Any	Any

Power off the ARM Cortex-M3 processor debug peripherals.

4.4.2.5 Read System ID Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x69 (‘i’)	`[0x69]`	*SysID*	Any	Any

Put the ARM Cortex-M3 processor debug port in the *SysID* transmit state so that the debug host can read the system identification information. For more information, see Section 4.4.2.6, “ARM Cortex-M3 Processor SysID Transmit State” on page 39.

4.4.2.6 ARM Cortex-M3 Processor SysID Transmit State

Syntax
[CM3 DEBUG VERSION MAJOR] [CM3 DEBUG VERSION MINOR] [CHIP FAMILY] [CHIP VERSION] [CHIP REVISION MAJOR] [CHIP REVISION MINOR]

This state is used to retrieve the system identification information. The 6-byte system identification response contains versioning information for the ARM Cortex-M3 core debug port protocol and the Ezairo 7100 chip. Send the **Read System ID** command to set up this response. It consists of the bytes described in the following subsections.

ARM Cortex-M3 Processor Debug Version

ARM Cortex-M3 core debug port protocol major version

The high byte of the value in the CFX mapped register `[CM3 DEBUG VERSION MAJOR]` at CFX address 0xF036. It is incremented when the ARM Cortex-M3 core debug port protocol is changed in a way that is not backwards compatible (and the ARM Cortex-M3 core debug port protocol minor version is reset to 0 as well). Changes that are not backwards compatible include (but are not limited to):

- Changing the ARM Cortex-M3 processor debug port response format
- Changing the byte sequence or format for a protocol command
- Changing the functionality of a command
- Removing support for a command

ARM Cortex-M3 core debug port protocol minor version

The low byte of the value in the CFX mapped register `[CM3 DEBUG VERSION MINOR]` at CFX address 0xF036. It is incremented when a backwards-compatible change is made to the ARM Cortex-M3 core debug port protocol.

If a future change to the debug protocol is backward-compatible with this version, the minor version number is updated. If a future change is not backward-compatible, the major version number is updated and the minor version number reset to 0.

ARM Cortex-M3 Processor Chip Identification

Ezairo 7100 chip family

This is the value in the CFX mapped register `[CHIP FAMILY]` at CFX address 0xF030. The `[CHIP FAMILY]` is identified in the *Hardware Reference Manual for Ezairo 7100* for your product.

Ezairo 7100 chip version

This is the value in the CFX mapped register `[CHIP VERSION]` at CFX address 0xF031.

Ezairo 7100 chip revision – major

This is the high byte of the value in the CFX mapped register `[CHIP REVISION MAJOR]` at CFX address 0xF032.

Ezairo 7100 chip revision – minor

This is the low byte of the value in the CFX mapped register `[CHIP REVISION MINOR]` at CFX address 0xF032. A change to the minor chip revision indicates that some internal functional change has been made to the chip that does not affect DSP application code.

The `[CHIP VERSION]`, `[CHIP REVISION MAJOR]` and `[CHIP REVISION MINOR]` are combined to identify a particular version of a chip. Different chip versions are assigned to identify drastically different and incompatible chips within a chip family. A change to the major chip revision indicates that some functional change has been made to the chip that might affect DSP application code.

Refer to the device identification section of the *Hardware Reference Manual for Ezairo 7100* for more information about major and minor revisions.

4.4.2.7 AHB-AP Overrun Clear Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x6F ('o')	`[0x6F]`	*Status*	Any	Any

Clear the AHB-AP command overrun flag that is part of the debug port status response.

All debug port commands except the following use AHB-AP commands to communicate with the ARM Cortex-M3 core:

- **Quit Debug**
- **Read Status**
- **AHB-AP Overrun Clear**
- **Read System ID**
- **Read and Reset CRC**
- **Config Coordinated Breakpoints**

If the debug port has to issue a new set of AHB-AP commands while the previous set of AHB-AP commands has not been completed, then the AHB-AP overrun flag is set. In this case the new set of AHB-AP commands will not be executed. This situation can occur if a debug command cannot complete in time due to a large number of AHB-AP commands, or when the ARM Cortex-M3 processor or one of the other processors in the Ezairo 7100 system is using a memory bus continuously for an extended period. An AHB-AP overrun can also occur when the ARM Cortex-M3 processor coordinated breakpoints mode is enabled and the CFX hits a breakpoint, as the ARM Cortex-M3 processor debug port will use the AHB-AP to perform the equivalent of a **Stop ARM Cortex-M3 Processor Core** command.

Whenever an overrun occurs, the overrun flag is set and the new command is not acknowledged. A user can check the AHB-AP busy bit of the debug port status response, to verify that the previous AHB-AP commands have finished. If an overrun occurs when reading or writing multiple values, the AHB-AP command that is currently in progress is aborted and any remaining data bytes transmitted are set to 0x1.

The AHB-AP command overrun flag is sticky, remaining set until it is cleared by the AHB-AP Overrun Clear ARM Cortex-M3 processor debug port command. The AHB-AP busy bit is not sticky; it is set for the whole duration of a debug port command's use of the AHB-AP (including the core register and memory commands when they are accessing multiple registers or memory locations) and is cleared otherwise.

4.4.2.8 Read and Reset CRC Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x6D (‘m’)	`[0x6D]`	*CRC*	Any	Any

Put the debug port in the *CRC* transmit state so that the debug host can read the debug port checksum. When the debug port executes this command, it stores the current CRC value to be sent to the debug host and resets the CRC value to 0xFFFF.

Use the **Read and Reset CRC** command to read the current CRC value and reset the CRC to 0xFFFF. The CRC includes all transferred bytes up to and including the **Read and Reset CRC** command. If the debug host reads the CRC using the next I^2C read transfer, the CRC bytes read by the debug host are included in the next CRC calculation.

The debug port maintains a CRC of all bytes transmitted and received. The CRC is updated automatically after a complete byte has been transferred in either direction. This includes all bytes that are not acknowledged, invalid commands, or commands attempted in the incorrect security or run mode. If a byte is not completely transferred, it is not included in the CRC. The CRC does not include the address byte of I^2C transactions.

Similar to the CFX debug port, the ARM Cortex-M3 processor debug port uses CRC-CCITT, which has the parameters described in Table 5 on page 24. For more information, see Section 4.4.2.9, “CRC Transmit State”.

4.4.2.9 CRC Transmit State

Syntax
`[CRC 15:8] [CRC 7:0]`

Transfer the value of the 16-bit debug port CRC when the preceding **Read and Reset CRC** command was executed. For more information, see Section 4.4.2.8, “Read and Reset CRC Command” on page 41.

4.4.3 Memory Access Commands and Responses

For memory transfers, the CFX DSP can be running. If the Ezairo 7100 security mode is restricted, the commands to read or write memory will not be acknowledged.

4.4.3.1 ARM Cortex-M3 Processor Memory Transfer Mode

Both memory access commands have a transfer mode parameter (1 byte) and an address parameter (4 bytes). The transfer mode byte contains the bitfields shown in Table 13.

IMPORTANT: Reading and writing odd addresses is not allowed in 16-bit data (half-word) mode. Similarly, reading and writing non-multiple of 4 addresses is not allowed in 32-bit data mode.

Table 13. Memory Transfer Mode

Bit(s)	Description	Values
7:6	Status Byte Frequency	00 - No status bytes 01 - Status byte is sent after every data value 10 - Status byte is sent after every second data value 11 - Status byte is sent after every fourth data value
5	Transfer Size	0 - Multiple value 1 - Single value
4	Address Increment	0 - Address not updated 1 - Address incremented after each value is transferred
3:2	Reserved	0
1:0	Data Width	00 - 8-bit 01 - 16-bit 10 - 32-bit 11 - Reserved

4.4.3.2 ARM Cortex-M3 Processor Read Memory Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x72 ('r')	`[0x72] [TRANSFER_MODE] [ADDR MSB] [ADDR 2] [ADDR 1] [ADDR LSB]`	*Memory*	Unrestricted	Any

Reading memory is initiated through the **Read Memory** debug port command. Reads the ARM Cortex-M3 processor memory through the AHB-AP.

The `[TRANSFER_MODE]` argument indicates the transfers:

- Width of a data value
- Transfer length (single or multi-value transfers)
- Addressing mode (incrementing or non-incrementing)
- How often status bytes are interleaved in the received data

Details of this argument are described in Section 4.4.3.1, "ARM Cortex-M3 Processor Memory Transfer Mode" on page 41. The next four bytes specify the starting address using big-endian byte ordering.

Once the Read Memory command has been sent, the debug port switches to the *Memory* transmit state.

To read the actual memory data, the debug host must start an I^2C read transfer. Depending on the setting of the *Address Increment* bit field, the address is automatically incremented after each successful transfer of a data value. The debug port performs no special processing on the address. The address automatically wraps around when it reaches the end of the 4 KB address range.

4.4.3.3 ARM Cortex-M3 Processor Memory Transmit State

Depending on the transfer mode, the memory transmit state follows one of the syntaxes listed below.

Status Byte Frequency Setting	Syntax
00	`[DATA.0] [DATA.1] ... [DATA.n]`
01	`[DATA.0] [STATUS] [DATA.1] [STATUS] ... [DATA.n] [STATUS]`
10	`[DATA.0] [DATA.1] [STATUS] ... [DATA.n] [STATUS]`[1]
11	`[DATA.0] [DATA.1] [DATA.2] [DATA.3] [STATUS] ... [DATA.n] [STATUS]`[2]

1. Final [STATUS] transfer is only transmitted for values of n that are multiples of 2
2. Final [STATUS] transfer is only transmitted for values of n that are multiples of 4

Transfers data from the debug port to the debug host in response to a **Read Memory** command. For more information, see Section 4.4.3.2, “ARM Cortex-M3 Processor Read Memory Command”.

4.4.3.4 ARM Cortex-M3 Processor Write Memory Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x77 (‘w’)	`[0x77] [TRANSFER_MODE] [ADDR MSB] [ADDR 2] [ADDR 1] [ADDR LSB] [DATA.0] [DATA.1] ... [DATA.n]`	*Status*	Unrestricted	Any

Writing memory is initiated through the **Write Memory** debug port command. Writes the ARM Cortex-M3 processor memory through the AHB-AP.

The `[TRANSFER_MODE]` argument indicates the transfers:

- Width of a data value
- Transfer length (single or multi-value transfers)
- Addressing mode (incrementing or non-incrementing)

Details of this argument are described in Section 4.4.3.1, “ARM Cortex-M3 Processor Memory Transfer Mode” on page 41. The next four bytes specify the starting address using big-endian byte ordering.

The Write Memory transaction occurs in a single I^2C write transfer. After the debug port has acknowledged the memory address, the debug host can begin sending data to be written to memory. Depending on the transfer mode, the debug host sends one, two, or four bytes per word. Once the full word has been transmitted, the debug port queues the word for writing to memory and acknowledges the last byte. Hence the debug host can send data to the debug port continuously without the debug port having to stretch the clock.

The debug port either automatically increments the address by one after each write or repeatedly writes to the same address. The debug port performs no special processing on the address. When using address increment mode, the address automatically wraps around when it reaches the end of the 4 kB address range.

4.4.4 Register Access Commands and Responses

The two categories of ARM Cortex-M3 processor registers are *core* registers and *AHB-AP* registers which have requirements as follows:

- Before you access the core registers of the ARM Cortex-M3 processor, the core must be stopped. The method by which the core is stopped does not matter (e.g., through an explicit stop, breakpoint, or a CFX controlled stop). If the core is not stopped, the commands to read or write core registers are not acknowledged.
- The registers for the AHB-AP component of the ARM Cortex-M3 processor subsystem can be accessed when the ARM Cortex-M3 core is running or stopped.

4.4.4.1 ARM Cortex-M3 Processor Read Core Register Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x63 ('c')	`[0x63] [TRANSFER CONTROL]`	*Register*	Unrestricted	Stopped

Put the debug port in the *Register* transmit state so that the debug host can read core registers. The transfer configuration depends on the Core Register Transfer Mode parameter, which provides the information described in Table 14.

Table 14. Core Register Transfer Mode

Bit(s)	Description	Values
7:6	Status Byte Frequency	00 - No status bytes 01 - Status byte is sent after every register 10 - Status byte is sent after every second register 11 - Status byte is sent after every fourth register
5	Transfer Size	0 - Multiple value 1 - Single value
4:0	Initial Register Index	Index of first register to send

The first register read from the ARM Cortex-M3 processor and sent to the I^2C master is identified by the start index. Subsequent bytes contain a sequence of register values. The ARM Cortex-M3 processor debug port increments the register index and begins transferring the register given by the new value of the index. When the register index reaches 31, it wraps back to 0.

Each register contains 4 bytes, and the debug port sends the most significant byte first. For registers that do not define all 32 of the bits in the register, the unused bits are zero padded. If a register index is reserved, the debug port sends zero.

The indexes are listed in Table 15.

Table 15. Core Register Indexes

Index	Register
0	R0
1	R1
2	R2
3	R3
4	R4
5	R5
6	R6
7	R7
8	R8
9	R9
10	R10
11	R11
12	R12
13	R13/current SP
14	R14/LR
15[1]	R15/PC
16	xPSR
17	Main SP (MSP)
18	Process SP (PSP)
19	*Reserved*
20[1]	CONTROL | FAULTMASK | BASEPRI | PRIMASK
21-31	*Reserved*

1. See the Debug Core Register Selector Register (DCRSR) in *The Definitive Guide to the ARM Cortex-M3* or a similar book about the ARM Cortex-M3 processor architecture.

4.4.4.2 ARM Cortex-M3 Processor Register Transmit State

Depending on the transfer mode, the register transmit state will follow one of the syntaxes listed below.

Status Byte Frequency Setting	Syntax
00	`[DATA.0] [DATA.1] ... [DATA.n]`
01	`[DATA.0] [STATUS] [DATA.1] [STATUS] ... [DATA.n] [STATUS]`
10	`[DATA.0] [DATA.1] [STATUS] ... [DATA.n] [STATUS]`[1]
11	`[DATA.0] [DATA.1] [DATA.2] [DATA.3] [STATUS] ... [DATA.n] [STATUS]`[2]

1. Final [STATUS] transfer is only transmitted for values of n that are multiples of 2

2. Final [STATUS] transfer is only transmitted for values of n that are multiples of 4

Transfers data from the debug port to the debug host in response to a **Read Core Registers** command. For more information, see Section 4.4.4.1, “ARM Cortex-M3 Processor Read Core Register Command”.

4.4.4.3 ARM Cortex-M3 Processor Write Core Register Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x66 ('f')	`[0x66] [TRANSFER CONTROL] [REG.0-MSB] [REG.0-2] [REG.0-1] [REG.0-LSB] ... [REG.n-MSB] [REG.n-2] [REG.n-1] [REG.n-LSB]`	*Status*	Unrestricted	Stopped

Write values to core registers. The transfer configuration depends on the Core Register Transfer Mode parameter that provides the information described in Table 14 on page 44, where the Transfer Size must be set for multiple values for proper operation.

The first register sent by the I^2C master and written to the ARM Cortex-M3 processor is identified by the start index. Subsequent bytes contain a sequence of register values. The ARM Cortex-M3 processor debug port increments the register index and begins transferring the register given by the new value of the index. When the register index reaches 31, it wraps back to 0. Each register contains 4 bytes, and the debug port sends the most significant byte first.

The **Write Core Register** command can supply any number of 4-byte values to store to ARM Cortex-M3 processor registers in a single I^2C write transfer. The ARM Cortex-M3 processor debug port can stretch the clock line if required while writing a register; however, if possible, the debug port queues the value for writing to the ARM Cortex-M3 processor register and acknowledges the last byte without waiting for the write to complete, thus allowing the I^2C master to send data to the ARM Cortex-M3 processor debug port continuously without the debug port having to stretch the clock. If during a new write transaction, the AHB is busy, the debug port issues a NAK.

Each register contains four bytes, and the debug port sends the most significant byte first. For registers that do not define all 32 bits in the register, the unused bits are zero padded. If a register index is reserved, the debug port is sent zeros.

The indexes are listed in Table 15 on page 45.

4.4.4.4 Read AHB-AP Register Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x6A ('j')	`[0x6A] [ADDRESS]`	*AHB Registers*	Unrestricted	Any

Put the debug port in the *AHB Registers* transmit state so that the debug host can read the AHB-AP registers. For more information on this state, see Section 4.4.4.5, “AHB-AP Register Transmit State”.

For more information about the registers accessible through the AHB access port, refer to the *Cortex-M3 Technical Reference Manual*.

The address provides the register read from the AHB-AP register and sent to the I^2C master. This command can access only a single register. After the four register bytes have been sent, most significant byte first, the ARM Cortex-M3 processor debug port sends zeros as long as the I^2C master continues to read and acknowledge the response bytes.

4.4.4.5 AHB-AP Register Transmit State

Syntax
`[DATA-MSB] [DATA-2][DATA-1][DATA-LSB]`

Transfers data from the debug port to the debug host in response to a **Read AHB-AP Registers** command. For more information, see Section 4.4.4.4, “Read AHB-AP Register Command”.

4.4.4.6 Write AHB-AP Register Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode

0x7A (‘z’)	`[0x7A] [ADDRESS]`
	`[VAL-MSB] [VAL-2]`
	`[VAL-1] [VAL-LSB]`
		Status	Unrestricted	Any

Write a single AHB access port register at the supplied address.

For more information about the registers accessible through the AHB access port, refer to the *ARM Debug Interface v5 Architecture Specification*.

The address provides the register sent by the I^2C master and written to the AHB-AP register. This command can access only a single register. After the four register bytes have been sent, most significant byte first, subsequent data are ignored.

4.4.5 Run Control Commands

4.4.5.1 Start ARM Cortex-M3 Core Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x67 (‘g’)	`[0x67]`	*Status*	Any	Any

Start the ARM Cortex-M3 core.

4.4.5.2 Stop ARM Cortex-M3 Core Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x70 ('p')	`[0x70]`	*Status*	Any	Any

Stops the ARM Cortex-M3 core.

4.4.5.3 ARM Cortex-M3 Processor Single-Step Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x74 ('t')	`[0x74]`	*Status*	Unrestricted	Stopped

Execute an instruction from the microcontroller application on the ARM Cortex-M3 core. When the debug port executes this command, the ARM Cortex-M3 core ignores any pending enabled interrupts.

This command executes the next instruction in the ARM Cortex-M3 processor program under debug. PendSV exceptions, SysTick exceptions, and external configurable interrupts are disabled, but the single-step might execute the exception entry sequence for a higher priority system exception either instead of or in addition to executing the next instruction of the program; see the *ARMv7-M Architecture Reference Manual* for details.

4.4.5.4 ARM Cortex-M3 Processor Single-Step with Interrupts Command

Command Byte	Syntax	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
0x6E ('n')	`[0x6E]`	*Status*	Unrestricted	Stopped

Execute an instruction from the microcontroller application on the ARM Cortex-M3 core. If interrupts are enabled on the ARM Cortex-M3 core and an enabled interrupt is pending when the debug port executes this command, the ARM Cortex-M3 core executes the interrupt vector operation instead of the instruction at the program counter. After the vector operation, the program counter points to the first instruction of the interrupt handler.

This command executes the next instruction in the ARM Cortex-M3 processor program under debug, or it executes the exception entry sequence for a pending exception (including interrupts), or it executes both if an exception becomes pending during the execution of the next instruction; see the *ARMv7-M Architecture Reference Manual* for details.

4.4.6 Breakpoint Commands and Responses

4.4.6.1 Coordinated Breakpoints Configuration Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x62 ('b')	`[0x62] [ENABLE]`	*Status*	Any	Any

Enable or disable coordinated breakpoints from the CFX. Enable is 1 and disable is 0.

The ARM Cortex-M3 processor debug port receives an input signal from the CFX debug port, which is asserted when the CFX hits a breakpoint. The Coordinated Breakpoints Configuration command enables or disables the ARM Cortex-M3 processor coordinated breakpoint mode, where a CFX breakpoint causes the ARM Cortex-M3 core to stop as well. When the ARM Cortex-M3 processor coordinated breakpoints mode is enabled, the assertion of a CFX breakpoint is equivalent to receiving the Stop ARM Cortex-M3 processor command when the ARM Cortex-M3 processor debug port is next idle and the transmit state is Transmit-Status. Since the stop occurs when the ARM Cortex-M3 processor debug port is idle, this stop never causes the AHB-AP overrun flag to be set.

The Coordinated Breakpoints Configuration command has a 1-byte parameter. The value of the parameter byte is interpreted as follows:

0	Disable ARM Cortex-M3 processor coordinated breakpoint mode
1	Enable ARM Cortex-M3 processor coordinated breakpoint mode
2 - 255	Reserved

APPENDIX A
JumpROM Function Quick Reference

Table 16. JumpROM functions

Function Index	JumpROM Function	Input	Output	Description
0x00	NOP	Not used	None	No operation
0x01	GetVersion	Not used	Program ROM version number	Load the Program ROM version into the debug port data register
0x02	Reset	Not used	None	Reset the system
0x03	Unlock	Not used	None	Verify secure EEPROM area is cleared, then clear memory and unlock the device
0x04	Wipe	AT_WREN opcode	Timeout status bit for EEPROM writes at bit position SR_X_ZERO (0 - No timeout, 1 - Timeout)	Clear the secure EEPROM area, then clear memory and unlock the device
0x05	-			Reserved
0x06	GetError	Not used	System error code	Load the current error number into the debug port data register
0x07	BootApp	Application to bootload	None	Boot the specified application
0x08	-			Reserved
0x09	Lock	Not used	None	Lock the device

APPENDIX B
CFX Debug Port Protocol Quick Reference

Table 17. Debug Port Protocol Commands

Command	Command Byte	Syntax and Description	Transmit State	Security Mode	CFX Run Mode
NOP	0x00	`[0x00]`	*Status*	Any	Any
Read Status	0x53 (‘S’)	`[0x53]`	*Status*	Any	Any
Write CFX Address	0x41 (‘A’)	`[0x41] [ADDRESS]`	*Status*	Any	Any
Write ARM Cortex-M3 Processor Debug Address	0x61 (‘a’)	`[0x61] [ADDRESS]`	*Status*	Any	Any
Acquire CFX Address	0x04	`[0x04] [MASK]`	*Status*	Any	Any
Reset CFX Debug Port	0x07	`[0x07]`	*Initialize*	Any	Any
Reset ARM Cortex-M3 Processor Debug Port	0x51 (‘Q’)	`[0x51]`	*Initialize*	Any	Any
Set Monitor Reset Bit	0x43 (‘C’)	`[0x43]`	*Status*	Any	Any
Read System ID	0x49 (‘I’)	`[0x49]`	*SysID*	Any	Any
Read and Reset CRC	0x4D (‘M’)	`[0x4D]`	*CRC*	Any	Any
Reset and Start Cycle Counter	0x58 (‘X’)	`[0x58]`	*Status*	Unrestricted	Any
Get Cycle Count	0x56 (‘V’)	`[0x56]`	*Cycle Count*	Unrestricted	Any
Read Memory	0x52 (‘R’)	`[0x52] [TRANSFER_MODE]` `[ADDR 15:8] [ADDR 7:0]`	*Memory*	Unrestricted	Stopped

Write Memory	0x57 (‘W’)	`[0x57] [TRANSFER_MODE]`
		`[ADDR 15:8] [ADDR 7:0]`
		`[DATA.0] [DATA.1] ... [DATA.n]`
			Status	Unrestricted	Stopped

Read Normal Registers	0x44 (‘D’)	`[0x44] [REGISTER INDEX]`	*Normal Registers*	Unrestricted	Stopped

Write Normal Registers	0x46 (‘F’)	`[0x46] [REGISTER INDEX]`
		`[REG.0 23:16] [REG.0 15:8]`
		`[REG.0 7:0] ... [REG.n 23:16]`
		`[REG.n 15:8] [REG.n 7:0]`
			Status	Unrestricted	Stopped

Read Special Registers	0x31 (‘1’)	`[0x31] [REGISTER INDEX]`	*Special Registers*	Unrestricted	Any

Write Special Registers	0x32 (‘2’)	`[0x32] [REGISTER INDEX]`
		`[REG.0 15:8] [REG.0 7:0] ...`
		`[REG.n 15:8] [REG.n 7:0]`
			Status	Unrestricted	Stopped

Read Debug Port Register	0x33 (‘3’)	`[0x33]`	*Debug Port Register*	Any	Any

Table 17. Debug Port Protocol Commands (Continued)

Command	Command Byte	Syntax and Description	Transmit State	Security Mode	CFX Run Mode
Write Debug Port Register	0x34 (‘4’)	[0x34] [DATA 31:24] [DATA 23:16] [DATA 15:8] [DATA 7:0]	*Status*	Any	Any
Start CFX core	0x47 (‘G’)	[0x47]	*Status*	Any	Any
Start CFX and ARM Cortex-M3 cores	0x55 (‘U’)	[0x55]	*Status*	Any	Any
Stop CFX core	0x50 (‘P’)	[0x50]	*Status*	Any	Any
Stop CFX and ARM Cortex-M3 cores	0x59 (‘Y’)	[0x59]	*Status*	Any	Any
CFX Single-step	0x54 (‘T’)	[0x54]	*Status*	Unrestricted	Stopped
CFX Single-step with Interrupts	0x4E (‘N’)	[0x4E]	*Status*	Unrestricted	Stopped
Execute Instruction	0x4F (‘O’)	[0x4F] [INSTR 31:24] [INSTR 23:16] [INSTR 15:8] [INSTR 7:0]	*Status*	Unrestricted	Stopped
JumpROM	0x4A (‘J’)	[0x4A] [CMD 7:0] [ARG 15:8] [ARG 7:0]	*Status*	Any	Any
Set Hardware Breakpoints	0x42 (‘B’)	[0x42] [HWBP.0] [ADDR.0 15:8] [ADDR.0 7:0] ... [HWBP.n] [ADDR.n 15:8] [ADDR.n 7:0] where: HWBP.n[7] – Enable (1)/Disable (0) BP HWBP.n[6:3] – Reserved HWBP.n[2:0] - Hardware BP Index (0-7) ADDR.n – Address for breakpoint	*Status*	Unrestricted	Any
Write Breakpoint Config	0x4B (‘K’)	[0x4B] [ESWBP] [HWBP] where: ESWBP[0] – Enable(1)/Disable(0) SW BPs ESWBP[1] – Enable(1)/Disable(0) Ext. BPs ESWBP[2] – Enable(1)/Disable(0) Coord. BPs HWBP[0] - Enable(1)/Disable(0) HW BP 0 HWBP[1] - Enable(1)/Disable(0) HW BP 1 … HWBP[7] - Enable(1)/Disable(0) HW BP 7	*Status*	Any	Any
Read Breakpoints	0x45 (‘E’)	[0x45]	*Breakpoints*	Unrestricted	Any

Table 18. Debug Port Protocol Transmit States

Transmit State	Syntax
Status	[STATUS 15:8] [STATUS 7:0] ... [STATUS 15:8] [STATUS 7:0]
SysID	[DEBUG PROTOCOL MAJOR] [DEBUG PROTOCOL MINOR] [CHIP FAMILY] [CHIP VERSION] [CHIP REVISION MAJOR] [CHIP REVISION MINOR]

Table 18. Debug Port Protocol Transmit States (Continued)

Transmit State	Syntax
CRC	[CRC 15:8] [CRC 7:0]
Cycle Count	[CYCLE COUNT 15:8] [CYCLE COUNT 7:0]
Memory	[DATA.0] [DATA.1] ... [DATA.n]
Normal Registers	[REG.0 23:16] [REG.0 15:8] [REG.0 7:0] ... [REG.n 23:16] [REG.n 15:8] [REG.n 7:0]
Special Registers	[REG.0 15:8] [REG.0 7:0] ... [REG.n 15:8] [REG.n 7:0]
Debug Port Register	[DATA 31:24] [DATA 23:16] [DATA 15:8] [DATA 7:0]
Breakpoints	[ESWBP] [HWBP] [ADDR.0 15:8] [ADDR.0 7:0] ... [ADDR.7 15:8] [ADDR.7 7:0]

Table 19. Debug Port Status Response

Bit	Status Byte Fields	Values[1]
15	High Byte Indicator	1 – Indicates high byte of status response
14	Coordinated Breakpoint Enable Status	0 – Coordinated breakpoints disabled 1 – Coordinated breakpoints enabled
13	I^2C Bus Error Bit[2]	1 – Bus error has occurred 0 – No bus errors detected
12	Monitor Reset Bit	1 – **Set Monitor Reset Bit** command received since last system reset 0 – Default value (cleared by system reset only)
11	Security Mode	0 – Unrestricted mode 1 – Restricted mode
10	External Breakpoint Enable Status	0 – External breakpoints disabled 1 – External breakpoints enabled
9	Hardware Breakpoint Enable Status	0 – All hardware breakpoints disabled 1 – At least one hardware breakpoint enabled
8	Software Breakpoint Enable Status	0 – Software breakpoints disabled 1 – Software breakpoints enabled
7	Low Byte Indicator	0 – Indicates low byte of status response
6	Reserved	0
5	Debug Port Busy	0 – Debug port idle 1 – Debug port busy[3]
4	External Breakpoint Hit Status	0 – External breakpoint not hit 1 – External breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
3	Hardware Breakpoint Hit Status	0 – Hardware breakpoint not hit 1 – Hardware breakpoint hit Cleared when the CFX DSP core is run or single-stepped.

Table 19. Debug Port Status Response (Continued)

Bit	Status Byte Fields	Values[1]
2	Software Breakpoint Hit Status	0 – Software breakpoint not hit 1 – Software breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
1:0	Run state	00 – Running 01 – Stopped 10 – Core halted (SLEEP) 11 - Reserved

1. The bit changes between states when a change in the debug port state is observed unless otherwise specified.

2. The I^2C Bus Error Bit indicates that an I^2C watchdog timeout has occurred. This bit is cleared when the debug port executes the **Reset Debug Port** command, or when the DSP application reads the `D_I2C_ERROR_CNT` register. See the *Hardware Reference Manual for Ezairo 7100* for more information.

3. The debug port is busy if the debug port is blocked from accessing memory due to a CFX and HEAR memory conflict. See the *Hardware Reference Manual for Ezairo 7100* for more information.

Table 20. Transfer Mode

Bit(s)	Description	Values
7:5	Reserved	000
4	Transfer Size	0 - Multiple words 1 - Single word
3:2	Memory Space	00 - Reserved 01 - X memory 10 - Y memory 11 - P memory
1:0	Data Width	00 - 8-bit 01 - 16-bit 10 - 24-bit 11 - 32-bit

Table 21. Normal Register Indexes

Index	Register	Width		Index	Register	Width
0	*Reserved*			32	`XCIRC0`	15
1	*Reserved*			33	`XCIRC1`	15
2	*Reserved*			34	`XCIRC2`	15
3	*Reserved*			35	`XCIRC3`	15
4	`X0`	24		36	`XCIRC4`	15
5	`X1`	24		37	`XCIRC5`	15
6	`Y0`	24		38	`XCIRC6`	15

Table 21. Normal Register Indexes (Continued)

Index	Register	Width		Index	Register	Width
7	Y1	24		39	XCIFC7	15
8	XP0	16		40	YCIRC0	15
9	XP1	16		41	YCIRC1	15
10	XP2	16		42	YCIRC2	15
11	XP3	16		43	YCIRC3	15
12	XP4	16		44	*Ignored*	
13	XP5	16		45	*Ignored*	
14	XP6	16		46	*Ignored*	
15	XP7	16		47	SP	16
16	XSTEP0	16		48	LR	16
17	XSTEP1	16		49	ILPC	16
18	XSTEP2	16		50	SR	16
19	XSTEP3	16		51	ILSR	16
20	XSTEP4	16		52	A0.L	24
21	XSTEP5	16		53	A1.L	24
22	XSTEP6	16		54	B0.L	24
23	XSTEP7	16		55	B1.L	24
24	YP0	16		56	A0.H	24
25	YP1	16		57	A1.H	24
26	YP2	16		58	B0.H	24
27	YP3	16		59	B1.H	24
28	YSTEP0	16		60	A0.E	8
29	YSTEP1	16		61	A1.E	8
30	YSTEP2	16		62	B0.E	8
31	YSTEP3	16		63	B1.E	8
				64-255	*Reserved*	

Table 22. Special Register Indexes

Index	Register	Width	Index	Register	Width
0	LE0	16	8	LC0	12
1	LE1	16	9	LC1	12
2	LE2	16	10	LC2	12
3	LE3	16	11	LC3	12
4	LS0	16	12	LP	3
5	LS1	16	13	PC (Read-Only)	16
6	LS3	16	14-255	Reserved	
7	LS3	16			

APPENDIX C

ARM Cortex-M3 Core Debug Port Protocol Quick Reference

Table 23. ARM Cortex-M3 Core Debug Port Protocol Commands

Command	Command Byte	Syntax and Description	Transmit State	Security Mode	ARM Cortex-M3 Processor Run Mode
Quit Debug	0x71 (‘q’)	[0x71]	*Initialize*	Any	Any
Read Status	0x73 (‘s’)	[0x73]	*Status*	Any	Any
Enable Debug	0x65 (‘e’)	[0x65]	*Status*	Any	Any
Disable Debug	0x64 (‘d’)	[0x64]	*Status*	Any	Any
Read System ID	0x69 (‘i’)	[0x69]	*SysID*	Any	Any
AHB-AP Overrun Clear	0x6F (‘o’)	[0x6F]	*Status*	Any	Any
Read and Reset CRC	0x6D (‘m’)	[0x6D]	*CRC*	Any	Any
Read Memory	0x72 (‘r’)	[0x72] [TRANSFER_MODE] [ADDR MSB] [ADDR 2] [ADDR 1] [ADDR LSB]	*Memory*	Unrestricted	Any
Write Memory	0x77 (‘w’)	[0x77] [TRANSFER_MODE] [ADDR MSB] [ADDR 2] [ADDR 1] [ADDR LSB] [DATA.0] [DATA.1] ... [DATA.n]	*Status*	Unrestricted	Any
Read Core Registers	0x63 (‘c’)	[0x63] [TRANSFER CONTROL]	*Register*	Unrestricted	Stopped
Write Core Registers	0x66 (‘f’)	[0x66] [TRANSFER CONTROL] [REG.0-MSB] [REG.0-2] [REG.0-1] [REG.0-LSB] ... [REG.n-MSB] [REG.n-2] [REG.n-1] [REG.n-LSB]	*Status*	Unrestricted	Stopped
Read AHB-AP Register	0x6A (‘j’)	[0x6A] [ADDRESS]	*AHB-Registers*	Unrestricted	Any
Write AHB-AP Registers	0x7A (‘z’)	[0x7A] [ADDRESS] [VAL-MSB] [VAL-2] [VAL-1] [VAL-LSB]	*Status*	Unrestricted	Any
Start core	0x67 (‘g’)	[0x67]	*Status*	Any	Any
Stop core	0x70 (‘p’)	[0x70]	*Status*	Any	Any
Single-step	0x74 (‘t’)	[0x74]	*Status*	Unrestricted	Stopped
Single-step with Interrupts	0x6E (‘n’)	[0x6E]	*Status*	Unrestricted	Stopped
Coordinated Breakpoints Configuration	0x62 (‘b’)	[0x62] [ENABLE]	*Status*	Any	Any

Table 24. Debug Port Status Response

Bit	Status Byte Fields	Values[1]
15	High Byte Indicator	1 – Indicates high byte of status response
14	Coordinated Breakpoint Enable Status	0 – Coordinated breakpoints disabled 1 – Coordinated breakpoints enabled
13:12	Reserved	00
11	Security Mode	0 – Unrestricted mode 1 – Restricted mode
10	ARM Cortex-M3 Processor Lockup Status	0 – The ARM Cortex-M3 processor is not in a lockup state 1 – The ARM Cortex-M3 processor is locked up
9	ARM Cortex-M3 Processor Enable Status	0 – The ARM Cortex-M3 processor is disabled 1 – The ARM Cortex-M3 processor is enabled
8	Reserved	000
7	Low Byte Indicator	0 – Indicates low byte of status response
6	Reserved	0
5	AHB-AP Command Busy	0 – No AHB-AP commands are not active 1 – An AHB-AP command is being executed
4	AHB-AP Overrun Status	0 – No AHP-AP command overruns have occurred 1 – An AHP-AP command overruns have occurred
3:2	Reserved	000
1:0	Run state	00 – Running 01 – Stopped 10 – Core halted (sleeping due to WFI, WFE, or Sleep on exit) 11 - Reserved

1. The bit changes between states when a change in the debug port state is observed unless otherwise specified.

Table 25. Memory Transfer Mode

Bit(s)	Description	Values
7:6	Status Byte Frequency	00 - No status bytes 01 - Status byte is sent after every data value 10 - Status byte is sent after every second data value 11 - Status byte is sent after every fourth data value
5	Transfer Size	0 - Multiple value 1 - Single value
4	Address Increment	0 - Address not updated 1 - Address incremented after each value is transferred

Table 25. Memory Transfer Mode (Continued)

Bit(s)	Description	Values
3:2	Reserved	0
1:0	Data Width	00 - 8-bit 01 - 16-bit 10 - 32-bit 11 - Reserved

Table 26. Core Register Transfer Mode

Bit(s)	Description	Values
7:6	Status Byte Frequency	00 - No status bytes 01 - Status byte is sent after every register 10 - Status byte is sent after every second register 11 - Status byte is sent after every fourth register
5	Transfer Size	0 - Multiple value 1 - Single value
4:0	Initial Register Index	- Index of first register to send

Table 27. Core Register Indexes

Index	Register
0	R0
1	R1
2	R2
3	R3
4	R4
5	R5
6	R6
7	R7
8	R8
9	R9
10	R10
11	R11
12	R12
13	R13/current SP
14	R14/LR
15[1]	R15/PC
16	xPSR
17	Main SP (MSP)
18	Process SP (PSP)
19	*Reserved*

Table 27. Core Register Indexes (Continued)

Index	Register
20[1]	CONTROL | FAULTMASK | BASEPRI | PRIMASK
21-31	*Reserved*

1. See the Debug Core Register Selector Register (DCRSR) from the ARMv7-M Architecture Reference Manual